SVELTE

Svelte v5

Svelte is a modern JavaScript framework for building user interfaces.
Unlike React or Vue, Svelte does its work at compile time
converting your components into highly optimized vanilla JavaScript,
rather than shipping a heavy runtime to the browser.
This means faster load times, smaller bundle sizes, and better performance.

Key Points:
Compiler-based — no virtual DOM, compiles to plain JavaScript
Less boilerplate — write less code compared to React or Vue
Reactive by default — state updates automatically update the UI
Scoped styles — CSS is scoped to the component by default
No runtime overhead — ships minimal JavaScript to the browser
SvelteKit — full-stack framework built on top of Svelte for routing, SSR, and more
Runes — Svelte 5 introduced runes ($state, $derived, $effect) for explicit reactivity

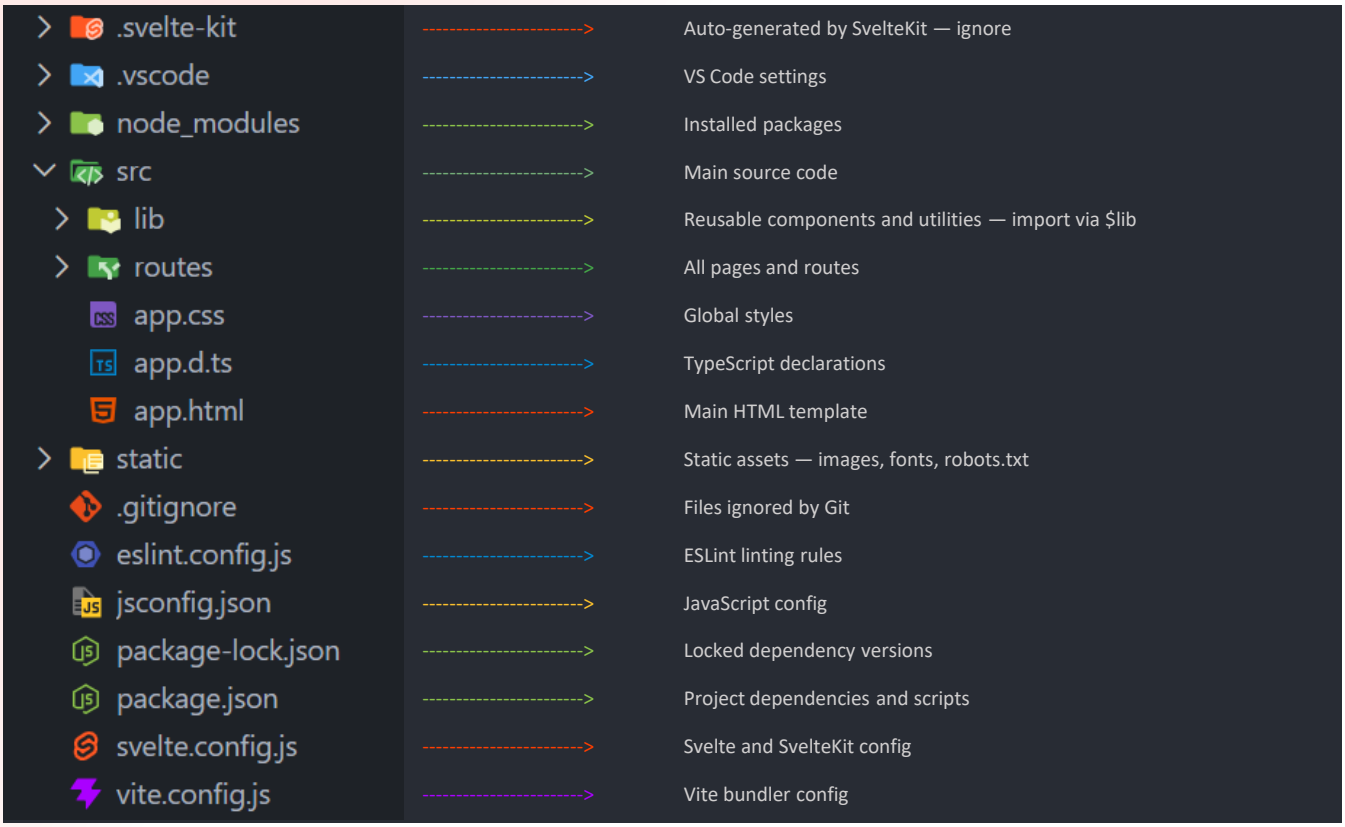
Svelte v5 : Notes Index

Svelte v5 : Notes Index - II

Svelte v5 : Installation & Folder Structure

1	npx sv create 1svelte	Install Svelte
	npm create vite@latest	Install Svelte using vite
2	cd 1svelte npm run dev	Move to project directory run server

Folder Structure in : SvelteKit v5



Svelte : Component Syntax

Components are the building blocks of Svelte applications. They are written into .svelte files, using a superset of HTML.
All three sections — script, styles and markup — are optional.

----> src/lib/components/Mycomponent.svelte

```svelte
<script>
    let name = 'Anjesh'; // In Script tag we can write JavaScript Code
</script>

<!-- In HTML tag we can write HTML Code and also we can use JavaScript by using {} -->
<h1>Hello {name}</h1>

<style>
    h1 {
        color: red; /* in Style tag we can write CSS Code */
    }
</style>
```

What is : Runes

Runes are symbols that you use in .svelte and .svelte.js / .svelte.ts files to control the Svelte compiler. If you think of Svelte as a language, runes are part of the syntax — they are keywords.

Runes have a $ prefix and look like functions:
They differ from normal JavaScript functions in important ways, however:

You don't need to import them — they are part of the language
They're not values — you can't assign them to a variable or pass them as arguments to a function
Just like JavaScript keywords, they are only valid in certain positions (the compiler will help you if you put them in the wrong place)

Runes : $state

$state is a reactive store that allows us to create reactive variables. When the value of count changes, the component will automatically re-render to reflect the new value.

-----> src/routes/+page.svelte

```
<script>

    let count = $state(0); // Create a reactive variable using $state Runes
</script>

<!-- increment button -->
<button onclick={() => count++}>Increment</button>

<!-- shows the current value of count -->
<p>Count : {count}</p>

<style></style>
```

State : Deep State

Objects and arrays are deeply reactive — mutating nested properties triggers UI updates automatically.

-----> src/routes/+page.svelte

```
<script>
    // Initialize todos state
    let todos = $state([
        { id: 1, done: false, text: 'Learn Svelte' },
        { id: 2, done: false, text: 'Eat Lunch' }
    ]);
</script>

<!-- Render each todo -->
{#each todos as todo (todo.id)}
    <p style:text-decoration={todo.done ? 'line-through' : 'none'}>{todo.text}</p>
    <button onclick={() => (todo.done = !todo.done)}>Toggle</button>
{/each}

<!-- Add a new todo -->
<button onclick={() => todos.push({ id: todos.length + 1, done: false, text: 'new todo' })}> Add Todo
</button>
```

```
<!-- Note: Destructuring breaks reactivity — always access properties directly. -->

let { done, text } = todos[0];    // ✕ not reactive

todos[0].done = !todos[0].done    // ✓ reactive
```

State : $state.raw — No Deep Reactivity

For large objects that are only reassigned, not mutated. Better performance.

-----> src/routes/+page.svelte

```
<script>
    let person = $state.raw({ name: 'Anjesh Kumar', age: 25 });
</script>

<!-- ✕ mutation — no effect -->
<button onclick={() => (person.age += 1)}>Mutate age (no effect)</button>

<!-- ✓ reassign — works -->
<button onclick={() => (person = { name: 'Ravi Kumar', age: 28 })}>Reassign person</button>

<!-- Render the name and age -->
<p>{person.name} - AGE {person.age}</p>
```

State : $state.snapshot — Proxy to Plain Object

Convert reactive proxy to plain object — useful for console.log or external libraries.

-----> src/routes/+page.svelte

```
<script>
    let counter = $state({ count: 0 });

    function onclick() {
        // logs plain object instead of Proxy
        console.log($state.snapshot(counter));
    }
</script>

<!-- Increment the count -->
<button onclick={() => counter.count++}> Increment </button>

<!-- Snapshot the count -->
<button {onclick}> snapshot to console</button>
<p>Count: {counter.count}</p>
```

State : Passing State into Functions

Pass getter functions to keep state reactive inside external functions useful when passing state to non-Svelte utilities or libraries.

-----> src/routes/+page.svelte

```
<script>
    let price = $state(100);  let tax = $state(10);

   // external function receives getter functions,
    function calculateTotal(getPrice, getTax) {
        return () => getPrice() + getTax(); // returns a function that always reads current values
    }
    // pass getter functions — () => price always returns current price
    let total = calculateTotal(
        () => price,
        () => tax );
</script>

<!-- increment price and tax -->
<button onclick={() => (price += 50)}>Increase Price: {price}</button>
<button onclick={() => (tax += 5)}>Increase Tax: {tax}</button>

<p>Total: {total()}</p>                <!-- call total() — returns current sum of price and tax -->
```

State : $state.eager — Immediate UI Update

By default async updates wait — $state.eager updates UI immediately. Use sparingly, only for user feedback.

-----> src/routes/+layout.svelte

```
<script>
    import favicon from '$lib/assets/favicon.svg';
    import { page } from '$app/state';                              // Import the page state
    let pathname = $derived($state.eager(page.url.pathname));       // Get the pathname
    let { children } = $props();                          // Get the props passed to the component
</script>

<!-- Render the favicon -->
<svelte:head><link rel="icon" href={favicon} /></svelte:head>

<nav><!-- Render the home link -->
    <a  href="/" aria-current={pathname === '/' ? 'page' : null}
        style:font-weight={pathname === '/' ? 'bold' : 'normal'}
    > Home </a>

   <!-- Render the about link -->
    <a  href="/about" aria-current={pathname === '/about' ? 'page' : null}
        style:font-weight={pathname === '/about' ? 'bold' : 'normal'}
    > About </a>
</nav>

{@render children()} <!-- Render children  -->

<!-- Style the navigation -->
<style>
    nav { display: flex; gap: 16px; padding: 16px; border-bottom: 1px solid #ccc; }
    a[aria-current='page'] { color: blue; font-weight: bold; }
</style>
```

```
<!-- file: src/routes/+page.svelte -->
<h1>Home</h1>
```

```
<!-- file: src/routes/about/+page.svelte -->
<h1>About</h1>
```

State : Sharing State Across Modules

Share reactive state across multiple components by declaring it in a .svelte.js

-----> src/routes/+page.svelte

```
<script>
    // Import the counter and functions
    import { counter, increment, decrement, reset } from '$lib/components/counter.svelte.js';
</script>

<!-- Render the count, and the buttons for increment, decrement, and reset -->
<h1>{counter.count}</h1>
<button onclick={increment}>Increment</button>
<button onclick={decrement}>Decrement</button>
<button onclick={reset}>Reset</button>
```

```
// src/lib/components/counter.svelte.js

export const counter = $state({ count: 0 });

export function increment() {   counter.count += 1;   }

export function decrement() {   counter.count -= 1;   }

export function reset()     {   counter.count = 0;    }
```

About : $derived

$derived is a rune in Svelte that computes a value from reactive state.
It automatically tracks its dependencies and recalculates whenever they change — no manual updates needed.

Unlike $effect, it does not cause side effects and should be used whenever you need to compute something from existing state.
For complex expressions, $derived.by accepts a function instead of a simple expression.

Runes : $derived

The value of doubleCount will automatically update whenever count changes

-----> src/routes/+page.svelte

```
<script>

    // Create a reactive variable using $state
    let count = $state(0);

    // Create a derived reactive variable using $derived
    let doubleCount = $derived(count * 2);

</script>

<!-- increment button -->
<button onclick={() => count++}>Increment</button>

<!-- shows the current value of count -->
<p>Count : {count}</p>
<p>Double Count : {doubleCount}</p>
```

Derived : $derived.by

$derived.by accepts a function as its argument. The function can contain multiple statements and can be as complex as needed.
The function will be re-run whenever any of the dependencies change, and the result will be cached until the next change.

-----> src/routes/+page.svelte

```
<script>

    // define a state variable 'number' with an initial array of numbers
    let number = $state([1, 2, 3, 4, 5]);

    //define a derived variable 'total' that calculates the sum of the numbers in the 'number' array
    let total = $derived.by(() => {
        let total = 0;
        for (const n of number) {
            total += n; // add each number in the 'number' array to the total
        }
        return total; // return the calculated total
    });
</script>

<!-- push a new number to the array and show the updated total -->
<button onclick={() => number.push(number.length + 1)}>
    {number.join(' + ')} = {total}
</button>
```

Derived : Deriveds and Reactivity

$derived values are not deeply reactive like $state — but if the source is $state, mutations still work.

-----> src/routes/+page.svelte

```
<script>
    let items = $state([{ name: 'apple' }, { name: 'banana' }]);
    let index = $state(0);

    // selected is derived from items
    let selected = $derived(items[index]);
</script>



<!-- change index -->
<button onclick={() => (index = 0)}>Apple</button>
<button onclick={() => (index = 1)}>Banana</button>


<!-- mutate selected — affects items directly -->
<button onclick={() => (selected.name = 'mango')}> Change to Mango </button>


<p>Selected: {selected.name}</p>
<p>Items: {items.map((i) => i.name).join(', ')}</p>
```

Derived : Destructuring

Destructuring a $derived keeps all variables reactive.

-----> src/routes/+page.svelte

```
<script>

   // returns a user object
    function getUser() {
        return { name: 'Anjesh', age: 25, role: 'Learner' };
    }

    // all three are reactive independently
    let { name, age, role } = $derived(getUser());
</script>

<!-- Render the user details -->
<p>Name: {name}</p>
<p>Age: {age}</p>
<p>Role: {role}</p>
```

Derived : Update propagation

If the new value of a derived is referentially identical to its previous value, updates will be skipped. In other words, Svelte will only update the text inside the button when large changes, not when count changes, even though large depends on count

-----> src/routes/+page.svelte

```
<script>
    let count = $state(0); // count is a state variable
   // large is a derived variable that depends on count. It will be true if count is greater than 10, and
   let large = $derived(count > 10);                                    false otherwise.

</script>
<button onclick={() => count++}> {large}</button>
```

Derived : untrack

Everything read synchronously inside $derived is tracked as a dependency. When any dependency changes, derived recalculates. To exempt state from being a dependency, use untrack:

----> src/routes/+page.svelte

```
<script>
    import { untrack } from 'svelte'; // Import the untrack function

    let count = $state(10);
    let multiplier = $state(4);

    // both count and multiplier are dependencies
    let result = $derived(count * multiplier);

    // only count is tracked — multiplier changes ignored
    let result2 = $derived(count * untrack(() => multiplier));
</script>

<!-- Render the result -->
<p>Result: {result}</p>
<p>Result2: {result2}</p>

<!-- Increment the count and multiplier -->
<button onclick={() => count++}>count: {count}</button>
<button onclick={() => multiplier++}>multiplier: {multiplier}</button>
```

Derived : Overriding Derived Values

Derived values can be temporarily overridden by reassigning them — useful for optimistic UI.

----> src/routes/+page.svelte

```
<script>
    let post = $state({ likes: 10 });
    let likes = $derived(post.likes); // derived state

    // show immediate feedback
    async function onclick() {
            likes += 1;

        try {
            // simulate server call
            await new Promise((f) => setTimeout(f, 1000));
        } catch {
            // failed — rollback
            likes -= 1;
        }
    }
</script>

<!-- Render the like button -->
<button {onclick}>  {likes}</button>
```

About : $effect

Effects are functions that automatically run when reactive state changes, and only execute in the browser — never during server-side rendering. Svelte tracks which reactive values are read inside an effect and re-Runes it when they change.
Multiple state changes are batched together, causing only one re-run. An effect can return a teardown function that Runes before the effect re-Runes or when the component is destroyed — useful for cleaning up intervals or event listeners. Use effects only for DOM manipulation, canvas drawing, or syncing with external libraries — never for synchronising state, that is what $derived is for.

Runes : $effect

$effect will automatically run whenever any of its dependencies change (in this case, count)

-----> src/routes/+page.svelte

```
<script>
    // Create a reactive variable using $state
    let count = $state(0);

    // Create a reactive effect using $effect
    $effect(() => {
        console.log(`Update count value is ${count}`);
    });
</script>

<!-- increment button -->
<button onclick={() => count++}>Increment</button>

<!-- shows the current value of count -->
<p>Count : {count}</p>

<style></style>
```

Effect : Understanding lifecycle

Return a cleanup function — Runes before effect re-Runes or component destroys.

-----> src/routes/+page.svelte

```
<script>
    let count = $state(0);
    let miliseconds = $state(1000);

    // effect Runes once when the component is mounted or re-rendered when dependencies change
    $effect(() => {
        const interval = setInterval(() => {
            count += 1;
        }, miliseconds);

        // cleanup function to be called when the effect is destroyed or unmounted
        // or when the component is re-rendered
        return () => {
            clearInterval(interval);
            console.log('effect destroyed or unmounted');
        };
    });
</script>

<!-- Render the count -->
<h1>{count}</h1>

<!-- Render the buttons to slow down or speed up the count -->
<button onclick={() => (miliseconds *= 2)}>slow</button>
<button onclick={() => (miliseconds /= 2)}>fast</button>
```

Effect : $effect.pending

When using await in components, the $effect.pending() rune tells you how many promises are pending in the current boundary, not including child boundaries: $effect.pending() return a number of pending promises if no any pending promises return 0

-----> src/routes/+page.svelte

```
<script>
    let a = $state(1);
    let b = $state(2);

    // async function
    async function add(a, b) {
        await new Promise((f) => setTimeout(f, 500)); // simulate delay
        return a + b;
    }
</script>

<!-- increment a and b on button click -->
<button onclick={() => a++}>a++</button>
<button onclick={() => b++}>b++</button>


<p>{a} + {b} = {await add(a, b)}</p>

<!-- Render the pending promises -->
{#if $effect.pending()}
    <p>pending promises: {$effect.pending()}</p>
{/if}
```

Effect : $effect.root

The $effect.root rune is an advanced feature that creates a non-tracked scope that doesn't auto-cleanup.
This is useful for nested effects that you want to manually control.
This rune also allows for the creation of effects outside of the component initialisation phase.

-----> src/routes/+page.svelte

```
<script>
    const destroy = $effect.root(() => {
        $effect(() => {
            console.log('effect running'); // setup Runes when dependencies change
        });

        return () => {
            console.log('cleaned up'); // clean-up Runes when destroy is called
        };
    });
</script>

<!-- manulay stop the effect later -->
<button onclick={() => destroy()}> Stop effects</button>
```

Effect : $effect.pre — Runes Before DOM Updates

In rare cases, you may need to run code before the DOM updates. For this we can use the $effect.pre rune:
Apart from the timing, $effect.pre works exactly like $effect.

-----> src/routes/+page.svelte

```
<script>
    import { tick } from 'svelte'; // Import the tick function

    let div = $state();             // Initialize the div
    let messages = $state([]);      // Initialize the messages array

    // Re-run the effect whenever the messages array changes before DOM updates
    $effect.pre(() => {
        if (!div) return;           // Wait for the div to be initialized
        messages.length;            // Trigger a re-run

        // Check if the div is at the bottom
        const isAtBottom = div.offsetHeight + div.scrollTop > div.scrollHeight - 50;
        if (isAtBottom) {
            return tick().then(() => div.scrollTo(0, div.scrollHeight)); // Scroll to the bottom
        }
    });
</script>

<!-- Render the messages -->
<div bind:this={div} style="height:200px; overflow-y:auto;">
    {#each messages as message (message)}
        <p>{message}</p>
    {/each}
</div>

<!-- Add a new message -->
<button onclick={() => messages.push(`Message ${messages.length + 1}`)}>Add Message</button>
```

Effect : $effect.tracking

The $effect.tracking rune is an advanced feature that tells you whether or not the code is running inside a tracking context, such as an effect or inside your template:

-----> src/routes/+page.svelte

```
<script>
    console.log('in component setup:', $effect.tracking()); // false

    $effect(() => {
        console.log('in effect:', $effect.tracking()); // true
    });
</script>

<p>in template: {$effect.tracking()}</p> <!-- true -->
```

Effect : Understanding Dependencies

$effect automatically tracks reactive values ($state, $derived, $props) read synchronously inside it and re-Runes when they change.

-----> src/routes/+page.svelte

```
1. Async values are not tracked
   Values read after await or inside setTimeout are ignored as dependencies.
-------------------------------------------------------------------------------
<script>
    $effect(() => {
        context.fillStyle = color; // ✅ tracked — re-Runes when color changes

        setTimeout(() => {
            context.fillRect(0, 0, size, size); // ❌ not tracked — size changes |ignored
        }, 0);
    });
</script>
-------------------------------------------------------------------------------



2. Object vs Property
   Effect reRunes when object itself changes — not when a property inside it mutates.
-------------------------------------------------------------------------------
<script>
  let state = $state({ value: 0 });
  let derived = $derived({ value: state.value * 2 });

  $effect(() => { state; });         // Runes once — object never reassigned
  $effect(() => { state.value; });   // Runes on every value change ✅
  $effect(() => { derived; });       // Runes every time — new object each time ✅
</script>

<button onclick={() => state.value += 1}>{state.value}</button>
<p>{state.value} doubled is {derived.value}</p>
-------------------------------------------------------------------------------



3. Conditional dependencies
   Effect only tracks values read in the last run.
-------------------------------------------------------------------------------
<script>
    let condition = $state(true);
    let color = $state('#ff3e00');

    $effect(() => {
        if (condition) {
            confetti({ colors: [color] }); // condition=true — both tracked
        } else {
            confetti(); // condition=false — only condition tracked, color ignored
        }
    });
</script>
-------------------------------------------------------------------------------
```

Effect : When not to use $effect

In general, $effect is best considered something of an escape hatch
useful for things like analytics and direct DOM manipulation — rather than a tool you should use frequently. In particular, avoid using it to synchronies state. Instead of this...

-----> src/routes/+page.svelte

❌ Don't — sync state with effect

```
<script>
  let count = $state(0);
  let doubled = $state();

  $effect(() => {
    doubled = count * 2;
  });
</script>
```

✅ Do — use $derived

```
<script>
  let count = $state(0);
  let doubled = $derived(count * 2);
</script>
```

❌ Don't — link values with effects

```
<script>
  const total = 100;
  let spent = $state(0);
  let left = $state(total);

  $effect(() => { left = total - spent; });
  $effect(() => { spent = total - left; });
</script>
```

✅ Do — use $derived and function bindings

```
<script>
  const total = 100;
  let spent = $state(0);
  let left = $derived(total - spent);

  function updateLeft(left) {
    spent = total - left;
  }
</script>

<input type="range" bind:value={spent} max={total} />
<input type="range" bind:value={() => left, updateLeft} max={total} />
```

Note: If you must update $state inside an effect and get an infinite loop — use untrack.

About : $props

Props are inputs passed to a component from its parent,
Just like attributes on HTML elements. Inside the component, props are received using the $props rune
and can be destructured for cleaner access. Props can have fallback values, be renamed, or collected as rest props.

They update automatically when the parent changes them — child can temporarily reassign but should never mutate props unless they are declared with $bindable

Runes : $props

$props is a special store that contains the props passed to the component
Create components folder in side “src/lib/” in component folder create Mycomponent.svelte file

-----> src/lib/components/Mycomponent.svelte

```
<script>
    let props = $props();
</script>

<p>This component is {props.adjective}</p>
```

Props : Pass props to Component

import and use Mycomponent from '$lib/components/Mycomponen.svelte'

-----> src/routes/+page.svelte

```
<script>
    // Import the component
    import Mycomponen from '$lib/components/Mycomponen.svelte';
</script>

<!-- Use the component and pass (adjective) props -->
<Mycomponen adjective="awesome" />
```

Props : Destructuring

we can destructure it to get the individual props this is recommended for better type inference and to avoid having to write $props.adjective every time we want to use it

-----> src/lib/components/Mycomponent.svelte

```
<script>
    // in this case, we are expecting an 'adjective' prop
    let { adjective } = $props();
</script>

<p>This component is {adjective}</p>
```

Props : Default props values

Destructuring allows us to declare fallback values, which are used if the parent component does not set a given prop (or the value is undefined): (in this example fallback value for adjective is 'Happy')

----> src/lib/components/Mycomponent.svelte

```
      // default value is used if the prop is not provided by the parent component

  let { adjective = 'Happy' } = $props();
```

Props : Renaming

We can also use the destructuring assignment to rename props, which is necessary if they're invalid identifiers,

----> src/lib/components/Mycomponent.svelte

```
      // Rename the super prop to trouper, and give it a default value

  let { super: trouper = 'lights are gonna find me' } = $props();
```

Props : Rest props

We can use a rest property to get, well, the rest of the props, Rest props are the ones that are not explicitly declared (a, b, c)

----> src/lib/components/Mycomponent.svelte

```
     // Store the rest of the props in a variable called 'others'

  let { a, b, c, ...others } = $props();
```

Props : $props.id()

Generates a unique ID for each component instance — consistent between server and client.
Useful for linking label and input elements.

----> src/lib/components/Mycomponent.svelte

```
  <script>
      const uid = $props.id(); // Generate a unique ID
  </script>

  <form>

      <!--  used to generate unique IDs -->
      <label for="{uid}-firstname">First Name:</label>
      <input id="{uid}-firstname" type="text" />

      <label for="{uid}-lastname">Last Name:</label>
      <input id="{uid}-lastname" type="text" />

  </form>
```

Props : Updating props

Props update in child when parent changes them. Child can temporarily reassign a prop but should never mutate it

1. Temporary reassign – allowed

[-----> src/routes/+page.svelte]

```svelte
<script>
    import Child from '$lib/components/Child.svelte';
    let count = $state(0);
</script>

<button onclick={() => (count += 1)}>
   clicks (parent): {count}
</button>

<Child {count} />
```

[-----> src/lib/components/Child.svelte]

```svelte
<script>
    let { count } = $props();
</script>

<!-- temporarily override – allowed -->
<button onclick={() => (count += 1)}>
  clicks (child): {count}
</button>
```

2. If the prop is a regular object, the mutation will have no effect:

[-----> src/routes/+page.svelte]

```svelte
<script>
    import Child from '$lib/components/Child.svelte';
</script>

<Child object={{ count: 0 }} />
```

[-----> src/lib/components/Child.svelte]

```svelte
<script>
  let { object } = $props();
</script>

<button
    onclick={() => {
        // has no effect
        object.count += 1;
    }}
>
    clicks {object.count}
</button>
```

Props : Updating props – II

3. If the prop is a reactive state proxy, however, then mutations will have an effect
   but you will see an ownership_invalid_mutation warning, because the component is mutating
   state that does not 'belong' to it:

[-----> src/routes/+page.svelte]

```svelte
<script>
    import Child from '$lib/components/Child.svelte';

    let object = $state({ count: 0 });
</script>

<Child {object} />
```

[-----> src/lib/components/Child.svelte]

```svelte
<script>
    let { object } = $props();
</script>

<button
    onclick={() => {
        // will cause the count below to update,
        // but with a warning. Don't mutate objects you don't own!
        object.count += 1;
    }}

> clicks: {object.count} |  </button>
```

4. Has no effect if the fallback value is used
   The fallback value of a prop not declared with $bindable is left untouched it is not
   turned into a reactive state proxy – meaning mutations will not cause updates:

[-----> src/routes/+page.svelte]

```svelte
<script>
    import Child from '$lib/components/Child.svelte';
</script>

<Child />
```

[-----> src/lib/components/Child.svelte]

```svelte
<script>
    let { object = { count: 0 } } = $props();
</script>

<button
    onclick={() => {
        // has no effect if the fallback value is used
        object.count += 1;
    }}

> clicks: {object.count} </button>
```

About : $bindable

Ordinarily, props go one way, from parent to child. This makes it easy to understand how data flows around your app.

In Svelte, component props can be bound, which means that data can also flow up from child to parent. This isn't something you should do often — overuse can make your data flow unpredictable and your components harder to maintain — but it can simplify your code if used sparingly and carefully.

It also means that a state proxy can be mutated in the child.

Runes : $bindable

In this example, we have a `FancyInput` component that accepts a `value` prop. By using `bind:value`, we can bind the value of the input to a variable in the parent component.

This means that when the user types into the input, the `message` variable in the parent component will automatically update with the new value.

-----> src/lib/components/FancyInput.svelte

```
<script>
    // Use bindable to create a bindable value for the component
    let { value = $bindable(), ...props } = $props();
</script>

<!--This is a simple input component that accepts a value and binds it to the input element. -->
<input type="text" bind:value {...props} />
```

Use Bind : in Component

-----> src/routes/+page.svelte

```
<script>
    // Import FancyInput component from the lib/components directory
    import FancyInput from '$lib/components/FancyInput.svelte';

   // Initialize message with a default value using $state for reactivity
    let message = $state('Hello World');
</script>

<!-- Bind the message state to the FancyInput component -->
<FancyInput bind:value={message} />

<!-- Display the message state -->
<p>{message}</p>
```

Default : Value

The parent component doesn't have to use bind: — it can just pass a normal prop. Some parents don't want to listen to what their children have to say.

In this case, you can specify a fallback value for when no prop is passed at all

-----> src/lib/components/FancyInput.svelte

```
<script>
    // Use bindable with default value
    let { value = $bindable('Default Value'), ...props } = $props();
</script>

<!--This is a simple input component that accepts a value and binds it to the input element. -->
<input type="text" bind:value {...props} />
```

About : $inspect

$inspect is a development-only debugging tool that works like console.log but automatically re-Runes whenever its reactive arguments change.

Unlike console.log which only logs once, $inspect tracks state deeply — meaning changes inside objects or arrays also trigger it. In production builds it does nothing.

Runes : $inspect — Development Only

Works like console.log but re-Runes whenever its argument changes.
Tracks reactive state deeply — only works in development, no-op in production.

-----> src/routes/+page.svelte

```svelte
<script>
    let count = $state(0);
    let message = $state('hello');

    $inspect(count, message); // will console.log when `count` or `message` change
</script>

<!-- render the button and input -->
<button onclick={() => count++}>Increment</button>
<input type="text" bind:value={message} />
```

Inspect : $inspect(...).with — Custom callback

returns an object with a with method, which you can invoke with a callback that will then be invoked instead of console.log. The first argument to the callback is either "init" or "update"; subsequent arguments are the values passed to $inspect:

-----> src/routes/+page.svelte

```svelte
<script>
    let count = $state(0);

    $inspect(count).with((type, count) => {
        if (type === 'update') {
            console.log('count changed to', count); // or `console.trace`, or whatever you want
        }
    });
</script>

<!-- render the button and input -->
<button onclick={() => count++}>Increment</button>
```

Inspect : $inspect.trace() — Trace dependencies

Traces which reactive state caused an effect or derived to re-run. Must be the first statement in the function body.

-----> src/routes/+page.svelte

```svelte
<script>
  import { doSomeWork } from './elsewhere';

  $effect(() => {
    $inspect.trace(); // must be first line
    doSomeWork();
  });
</script>
```

About : $host

$host is a rune that provides access to the host element when a Svelte component is compiled as a custom element.

It allows you to interact with the host DOM element directly — for example,
Dispatching custom events that the parent can listen to.

It is only available when using the customElement compiler option and has no effect in regular Svelte components.

Runes : $host

When compiling a component as a custom element, the $host rune provides access to the host element,

-----> src/routes/+page.svelte

```svelte
<script>
    import '$lib/components/Stepper.svelte'; // Import the Stepper component

    let count = $state(0); // Initialize count state
</script>


<!-- Bind the decrement and increment events -->
<my-stepper
ondecrement={() => (count -= 1)}
onincrement={() => (count += 1)}
>
</my-stepper>

<!-- Render the count -->
<p>count: {count}</p>
```

-----> src/lib/components/Stepper.svelte

```svelte
<!-- Register the custom element -->
<svelte:options customElement="my-stepper" />


<script>
    // Dispatch custom events
    function dispatch(type) {
        $host().dispatchEvent(new CustomEvent(type));
    }
</script>


<!-- Render the buttons -->
<button onclick={() => dispatch('decrement')}>decrement</button>
<button onclick={() => dispatch('increment')}>increment</button>


Note: $host only works when compiling with customElement option — not in regular Svelte components.
```

What is : Markup

Markup inside a Svelte component can be thought of as HTML++.

HTML : Tags

A lowercase tag, like , denotes a regular HTML element. A capitalised tag or a tag that uses dot notation, such as <Widget> or <my.stuff>, indicates a component.

-----> Example

```
<script>
    import Widget from './Widget.svelte';
</script>

<div>
    <Widget />
</div>
```

Element : attributes

By default, attributes work exactly like their HTML counterparts.

-----> Example

```
<div class="foo">
    <button disabled>can't touch this</button>
</div>

<!-- As in HTML, values may be unquoted. -->

<input type="checkbox" />

<!-- Attribute values can contain JavaScript expressions. -->

<a href="page/{p}">page {p}</a>

<!-- Or they can be JavaScript expressions -->
 <button disabled={!clickable}>...</button>
```

Short hand : attributes

By convention, values passed to components are referred to as properties or props rather than attributes, which are a feature of the DOM.

As with elements, name={name} can be replaced with the {name} shorthand.

-----> Example

```
<button {disabled}>...</button>
<!-- equivalent to <button disabled={disabled}>...</button> --
```

Spread : attributes

Spread attributes allow many attributes or properties to be passed to an element or component at once.

An element or component can have multiple spread attributes, interspersed with regular ones. Order matters — if things.a exists it will take precedence over a="b", while c="d" would take precedence over things.c:

-----> Example

```
<!-- separate the things object using the spread operator -->
<Widget a="b" {...things} c="d" />
```

About : Events

Event attributes are case sensitive. onclick listens to the click event, onClick listens to the Click event, which is different. This ensures you can listen to custom events that have uppercase characters in them.

Because events are just attributes, the same rules as for attributes apply:

you can use the shorthand form: <button {onclick}>click me</button>
you can spread them: <button {...thisSpreadContainsEventAttributes}>click me</button>
Timing-wise, event attributes always fire after events from bindings (e.g. oninput always fires after an update to bind:value).
Under the hood, some event handlers are attached directly with addEventListener, while others are delegated.

When using ontouchstart and ontouchmove event attributes, the handlers are passive for better performance. This greatly improves responsiveness by allowing the browser to scroll the document immediately, rather than waiting to see if the event handler calls event.preventDefault().

In the very rare cases that you need to prevent these event defaults, you should use on instead (for example inside an action).

Dom : Events

Listening to DOM events is possible by adding attributes to the element that start with on. For example, to listen to the click event, add the onclick attribute to a button:

----> Example

```
<!--This is a simple Svelte component that includes a button with an onclick event handler. When the button is clicked, it will log 'clicked' to the console. -->

<button onclick={() => console.log('clicked')}>click me</button>
```

Event : delegation

To reduce memory footprint and increase performance, Svelte uses a technique called event delegation. This means that for certain events — see the list below — a single event listener at the application root takes responsibility for running any handlers on the event's path.

There are a few gotchas to be aware of:

when you manually dispatch an event with a delegated listener, make sure to set the { bubbles: true } option or it won't reach the application root
when using addEventListener directly, avoid calling stopPropagation or the event won't reach the application root and handlers won't be invoked. Similarly, handlers added manually inside the application root will run before handlers added declaratively deeper in the DOM (with e.g. onclick={...}), in both capturing and bubbling phases. For these reasons it's better to use the on function imported from svelte/events rather than addEventListener, as it will ensure that order is preserved and stopPropagation is handled correctly.

Available : Events

beforeinput
click
change
dblclick
contextmenu
focusin
focusout
input
keydown
keyup

beforeinput
click
change
dblclick
contextmenu
focusin
focusout
input
keydown
keyup

Text : expressions

A JavaScript expression can be included as text by surrounding it with curly braces. **{expression}**

Expressions that are null or undefined will be omitted; all others are coerced to strings.

Curly braces can be included in a Svelte template by using their HTML entity strings: {, {, or { for { and }, }, or } for }.

If you're using a regular expression (RegExp) literal notation, you'll need to wrap it in parentheses.

-----> Example

```
<!-- Text Expressions -->
<h1>Hello {name}!</h1>

<!-- Variables Expressions -->
<p>{a} + {b} = {a + b}.</p>

<!-- Regular Expression & Conditional Expressions -->
<div>{(/^[A-Za-z ]+$/).test(value) ? x : y}</div>
```

About : Comments

You can use HTML comments inside components.

Comments beginning with svelte-ignore disable warnings for the next block of markup. Usually, these are accessibility warnings; make sure that you're disabling them for a good reason.

-----> Example

```
<!-- this is a comment! -->
<h1>Hello world</h1>

<!-- svelte-ignore a11y_autofocus -->
<input bind:value={name} autofocus />
```

Special comment : @component

You can add a special comment starting with @component
That will show up when hovering over the component name in other files.

-----> Example

```
<!--
@component
- You can use markdown here.
- You can also use code blocks here.
- Usage:
  ```html
  <Main name="Arethra">
  ```
-->
<script>
    let { name } = $props();
</script>

<main>
    <h1>
        Hello, {name}
    </h1>
</main>
```

About : {#if ...}

{#if} is a template block in Svelte used to conditionally render markup.

It supports {:else} for a fallback and {:else if} for multiple conditions — working exactly like JavaScript if, else if, and else statements but directly in the template.

Logic : if Block

This example of if logic block conditionally renders a login or logout button based on the user's logged-in state. The user state is initialized with a logged In property set to false, and the toggle function toggles this state when the button is clicked.

-----> src/routes/+page.svelte

```
<script>

    // Initialize the user state with a loggedIn property set to false
    let user = $state({ loggedIn: false });

    function toggle() {
      user.loggedIn = !user.loggedIn; //Toggle the loggedIn state using the not operator "!"
    }

</script>

<!-- If user is Logged in show logout button by toggle the loggedIn state as true -->

{#if user.loggedIn}
    <button onclick={toggle}> Log out </button>

{/if}

<!--If user is not logged in, show login button by toggle the  loggedIn state as false using not operator "!"

{#if user.loggedIn}                                                                    -->
    <button onclick={toggle}> Log In </button>

{/if}
```

Logic : if-else Block

Example of if-else statement in Svelte to conditionally render buttons based on the user's logged-in state.

-----> src/routes/+page.svelte

```
<script>
    // Initialize the user state with a loggedIn property set to false
    let user = $state({ loggedIn: false });

    function toggle() {
      user.loggedIn = !user.loggedIn; //Toggle the loggedIn state using the not operator "!"
    }


</script>

<!-- If loggedIn is true, show logout button else show login button -->

{#if user.loggedIn}
    <button onclick={toggle}> Log out </button>

{:else}
    <button onclick={toggle}> Log In </button>

{/if}
```

Logic Block : if-else-if

Example of else-if logic block show different messages based on the value of count

----> src/routes/+page.svelte

```
<script>


    let count = $state(0); //Initial value of count is 0


</script>

<!-- A simple counter app -->

<button onclick={() => count++}>Increment</button>



{#if count > 10}

    <p>{count} is greater than 10</p>

{:else if 5 > count}

    <p>{count} is less than 5</p>

{:else}

    <p>{count} is between 5 and 10</p>

{/if}
```

Conditional Rendering : Ternary Operator

Use ternary directly in template for simple inline conditions.

----> src/routes/+page.svelte

```
<script>
  let isLoggedIn = $state(false);
  let score = $state(75);
  let theme = $state('dark');
</script>


<!-- text -->
<p>{isLoggedIn ? 'Welcome back!' : 'Please log in'}</p>



<!-- class -->
<p class={score >= 50 ? 'pass' : 'fail'}>Score: {score}</p>



<!-- style -->
<div style:background={theme === 'dark' ? '#000' : '#fff'}>
  {theme}
</div>
```

Loop Block : each

{#each} is Svelte's way to loop through arrays, objects, or iterables like Map and Set.{#each} is Svelte's way to loop through arrays, objects, or iterables like Map and Set. It supports index tracking, destructuring, and an {:else} block that shows when the list is empty. Always provide a unique key so Svelte can efficiently add, move, or remove items without re-rendering the whole list.

-----> src/routes/+page.svelte

```
<script>
    let fruits = [                                      // Fruit object array
        { id: 1, name: 'Apple', color: 'Red' },
        { id: 2, name: 'Banana', color: 'Yellow' },
        { id: 3, name: 'Mango', color: 'Orange' },
    ];
</script>

| 1. Basic Showing the fruits object values
|-------------------------------------------------------------------------------
    {#each fruits as fruit}
        <li>{fruit.name} - {fruit.color}</li>
    {/each}
|-------------------------------------------------------------------------------

| 2. With index An each block can also specify an index, equivalent to the second argument
|-------------------------------------------------------------------------------
    {#each fruits as fruit, i}
        <li>Index: {i + 1} {fruit.name} - {fruit.color}</li>
    {/each}
|-------------------------------------------------------------------------------

| 3. With keyed – always use for better performance
|    A key uniquely identifies each list item – Svelte uses it to efficiently
|    insert, move, and delete items. Use strings or numbers as keys for best results.
|-------------------------------------------------------------------------------
    {#each fruits as fruit (fruit.id)}
        <li>Id : {fruit.id} {fruit.name} - {fruit.color}</li>
    {/each}
|-------------------------------------------------------------------------------

| 4. Destructuring
|-------------------------------------------------------------------------------
    {#each fruits as { id, name, color }, i (id)}
        <li>Index : {i + 1} {name} - {color}</li>
    {/each}
|-------------------------------------------------------------------------------

| 5. With Else blocks
|     An each block can also have an {:else} clause, which is rendered if the list is empty.
|-------------------------------------------------------------------------------
    {#each fruits as fruit (fruit.id)}
        <li>Id : {fruit.id} {fruit.name} - {fruit.color}</li>
    {:else}
        <p>No Items Found!</p>
    {/each}
|-------------------------------------------------------------------------------

| 5. Each blocks without an item
|    In case you just want to render something n times, you can omit the as part:
|-------------------------------------------------------------------------------
    {#each { length: 5 }, i}
        <p>Item {i + 1}</p>
    {/each}
|-------------------------------------------------------------------------------
```

Block : Key

Key blocks destroy and recreate their contents when the value of an expression changes.
When used around components, this will cause them to be reinstantiated and reinitialized:

In this #key block example we have a Count component that increments a count every second. The #key block will re-render the Count component whenever the 'reset' variable changes, effectively resetting the count to zero when the button is clicked.

-----> src/routes/+page.svelte

```
<script>
    import Count from '$lib/components/Count.svelte'; // Import the Count component

    let reset = $state(false); // Initialize reset state
</script>

{#key reset}
    <Count /> <!-- Render the Count component, it will reset when 'reset' changes -->
{/key}

<!-- Button to toggle reset state -->
<button onclick={() => (reset = !reset)}> Reset! </button>
```

Component : count

-----> src/lib/components/count.svelte

```
<script>
    let count = $state(0);     // Initialize count state
    setInterval(() => {
        count += 1;            // Increment count every second
    }, 1000);                  // Increment count every second
</script>

<h1>{count}</h1>               <!-- Render the count -->
```

Key : Example with Transition

-----> src/routes/+page.svelte

```
<script>
    import { fly } from 'svelte/transition'; // Import the fly transition from Svelte transition

    let count = $state(0); // Initialize a count variable
</script>

<!-- Render the count in #key block using the fly transition -->
<div class="counter-container">
    {#key count}
        <h1 in:fly={{ y: 20, duration: 300 }} out:fly={{ y: -20, duration: 300 }}>
            {count}
        </h1>
    {/key}

    <!-- Button to increment the count -->
    <button onclick={() => count++}> Next Number </button>
</div>

<style>
    .counter-container { display: grid; place-items: center; height: 100px; margin-top: 20px; }
    h1 { grid-area: 1 / 1; font-size: 4rem; margin: 0; color: #ff3e00; }
</style>
```

Logic Block : await

The {#await} block is a Svelte syntax for handling promises. It allows you to show different content based on the state of the promise (loading, resolved, or rejected). In this example, we fetch data from an API and display it once it's loaded, while showing a loading message in the meantime. If there's an error during the fetch, it will display the error message.

----> src/routes/+page.svelte

```
<script>
    function fetchData() {     // Fetch data from an API and return a promise
        let users = fetch('https://jsonplaceholder.typicode.com/posts/1')
                    .then((res) => res.json());
        return users;
    }
</script>

<h1>Fetched Data from API</h1>

{#await fetchData()}           <!-- call the function that returns a promise  -->
    <p>Loading...</p>          <!--while waiting for the promise to resolve, show a loading message -->

{:then data}                   <!-- once the promise resolves, we can access the data -->
    <p>{data.title}</p>
    <p>{data.body}</p>         <!-- display the title and body from the fetched data -->

    {:catch error}             <!-- if there is an error while fetching the data, catch it -->

        <p>Error: {error}</p> <!-- display the error message -->
{/await}

| The catch block can be omitted if you don't need to render anything when
| the promise rejects (or no error is possible)
|-------------------------------------------------------------------------
    {#await promise}
    <!-- promise is pending -->
    <p>waiting for the promise to resolve...</p>
    {:then value}
    <!-- promise was fulfilled -->
    <p>The value is {value}</p>
    {/await}
|-------------------------------------------------------------------------


| If you don't care about the pending state, you can also omit the initial block.
|-------------------------------------------------------------------------
    {#await promise then value}
    <p>The value is {value}</p>
    {/await}
|-------------------------------------------------------------------------


| Similarly, if you only want to show the error state, you can omit the then block.
|-------------------------------------------------------------------------
    {#await promise catch error}
    <p>The error is {error}</p>
    {/await}
|-------------------------------------------------------------------------


| You can use #await with import(...) to render components lazily:
|-------------------------------------------------------------------------
    {#await import('./Component.svelte') then { default: Component }}
    <Component />
    {/await}
|-------------------------------------------------------------------------
```

Syntax : #snippet

Snippets, and render tags, are a way to create reusable chunks of markup inside your components.
Instead of writing duplicative code you can write this, To render a snippet, use a {@render ...} tag

-----> src/routes/+page.svelte

```
<script>
    let images = [
        {
            id: 1,
            src: 'https://picsum.photos/600/200',
            width: 600,
            height: 200,
            caption: 'this image has a link',
            href: 'https://example.com'
        },
        {
            id: 2,
            src: 'https://picsum.photos/600/250',
            width: 600,
            height: 250,
            caption: 'this one does not'
        }
    ];
</script>

<h1>photos</h1>

<!-- 1 Define snippet with parameter in this case (image) -->

{#snippet figure(image)}
    <figure>
        <img src={image.src} alt={image.caption} width={image.width}
                                                        height={image.height} />
        <figcaption>{image.caption}</figcaption>
    </figure>
{/snippet}


<!-- 2 Use snippet inside this loop -->

{#each images as image (image.id)}
    {#if image.href}                    <!--Check if the image has a link -->
        <a href={image.href}>           <!-- If it does, wrap the figure in an anchor tag -->
            {@render figure(image)}     <!-- To render a snippet, use a {@render ...} tag -->
        </a>
    {:else}
        {@render figure(image)}         <!-- 4 Otherwise, just render the figure -->
    {/if}
{/each}

<style>
    h1 { font-size: 2em;}

    figure {
        background: white;
        padding: 1em;
        margin: 0 0 1em 0;
        filter: drop-shadow(2px 4px 10px rgba(0, 0, 0, 0.1));
    }

    img { width: 100%; height: auto; background: #eee; }
</style>
```

Note :
Like function declarations, snippets can have an arbitrary number of parameters, which can have default values, and you can destructure each parameter. You cannot use rest parameters, however.

Snippets : Explicit props

Within the template, snippets are values just like any other. As such, they can be passed to components as props:
Think about it like passing content instead of data to a component. The concept is similar to slots in web components.

----> src/routes/+page.svelte

```svelte
<script>
    import Table from '$lib/components/Table.svelte';
    // Define the data for the table
    const fruits = [
        { name: 'apples', qty: 5, price: 2 },
        { name: 'bananas', qty: 10, price: 1 },
        { name: 'cherries', qty: 20, price: 0.5 }
    ];
</script>

<!-- Define the header snippets -->
{#snippet header()}
    <th>fruit</th>
    <th>qty</th>
    <th>price</th>
    <th>total</th>
{/snippet}

<!-- Define the row snippets -->
{#snippet row(d)}
    <td>{d.name}</td>
    <td>{d.qty}</td>
    <td>{d.price}</td>
    <td>{d.qty * d.price}</td>
{/snippet}

<!-- Render the table -->
<Table data={fruits} {header} {row} />
```

Table : Component

----> src/lib/components/Table.svelte

```svelte
<script>
    let { data, header, row } = $props(); // Get the props passed to the component
</script>

    <table>
        <!-- Render the header -->
        {#if header}
            <thead>
                <tr> {@render header()}</tr>
            </thead>
        {/if}

        <!-- Render the rows -->
        {#if data}
            <tbody>
                {#each data as d, i (i)}
                    <tr>{@render row(d)}</tr>
                {/each}
            </tbody>
        {/if}
    </table>

<style>
    table {text-align: left;border-spacing: 0;}
    tbody tr:nth-child(2n + 1) {background: #f0f0f0;}
    table :global(th),table :global(td) {padding: 0.5em;}
</style>
```

Snippets : Implicit props

As an authoring convenience, snippets declared directly inside a component implicitly become props on the component:

----> src/routes/+page.svelte

```
<script>
    import Table from '$lib/components/Table.svelte';
    // Define the data for the table
    const fruits = [
        { name: 'apples', qty: 5, price: 2 },
        { name: 'bananas', qty: 10, price: 1 },
        { name: 'cherries', qty: 20, price: 0.5 }
    ];
</script>

<!-- Render the table -->
<Table data={fruits}>
    <!-- Define the header snippets -->
    {#snippet header()}
        <th>fruit</th>
        <th>qty</th>
        <th>price</th>
        <th>total</th>
    {/snippet}

    <!-- Define the row snippets -->
    {#snippet row(d)}
        <td>{d.name}</td>
        <td>{d.qty}</td>
        <td>{d.price}</td>
        <td>{d.qty * d.price}</td>
    {/snippet}
</Table>
```

Table : Component

----> src/lib/components/Table.svelte

```
<script>
    let { data, header, row } = $props(); // Get the props passed to the component
</script>

    <table>
        <!-- Render the header -->
        {#if header}
            <thead>
                <tr> {@render header()}</tr>
            </thead>
        {/if}

        <!-- Render the rows -->
        {#if data}
            <tbody>
                {#each data as d, i (i)}
                    <tr>{@render row(d)}</tr>
                {/each}
            </tbody>
        {/if}
    </table>

<style>
    table {text-align: left;border-spacing: 0;}
    tbody tr:nth-child(2n + 1) {background: #f0f0f0;}
    table :global(th),table :global(td) {padding: 0.5em;}
</style>
```

Snippets : Implicit children

Any content inside the component tags that is not a snippet declaration implicitly becomes part of the children snippet

-----> src/routes/+page.svelte

```
<script>
    // 1 Import Button component
    import Button from '$lib/components/Button.svelte';
</script>

<!-- 2 Render Button -->
<Button>Click me</Button>
```

[-----> src/lib/components/Button.svelte]

```
<script>
    let { children } = $props();
</script>

<!- This button component that accepts children and renders them inside the button element. -->
<button>{@render children()}</button>
```

Exporting : Snippet

Snippets declared at the top level of a .svelte file can be exported from a <script module> for use in other components, provided they don't reference any declarations in a non-module <script> (whether directly or indirectly, via other snippets):

-----> src/routes/+page.svelte

```
<script>
    // Import add snippet from snippets file
    import { add } from '$lib/components/snippets.svelte';
</script>

<!-- Render add snippet -->
{@render add(1, 2)}
```

[-----> src/lib/components/snippets.svelte]

```
<script module>        // use module

      export { add }; //export snippet
</script>

<!-- Define add snippet -->
{#snippet add(a, b)}
    {a} + {b} = {a + b}
{/snippet}
```

Render : Snippet

To render a snippet, use a {@render ...} tag.

-----> src/routes/+page.svelte

```
<!-- Define a snippet -->
{#snippet sum(a, b)}
    <p>{a} + {b} = {a + b}</p>
{/snippet}

<!-- Render the snippet -->
{@render sum(1, 2)}
{@render sum(3, 4)}
{@render sum(5, 6)}
```

```
The expression can be an identifier like sum, or an arbitrary JavaScript expression:
Render cool snippet if cool is true, else render lame snippet
-------------------------------------------------------------------------------
   {@render (cool ? coolSnippet : lameSnippet)()}

-------------------------------------------------------------------------------
```

Optional : Snippet props

You can declare snippet props as being optional. You can either use optional chaining to not render anything if the snippet isn't set. or use an #if block to render fallback content:

-----> src/routes/+page.svelte

```
Optional rendering using “?”
-------------------------------------------------------------------------------
    <script>
        let { children } = $props();
    </script>

    {@render children?.()}
-------------------------------------------------------------------------------
```

```
Optional rendering using if-else block
-------------------------------------------------------------------------------
    <script>
        let { children } = $props();
    </script>

    {#if children}
         {@render children()}
    {:else}
        fallback content
    {/if}
-------------------------------------------------------------------------------
```

Snippet : scope

Snippets can be declared anywhere inside your component. They can reference values declared outside themselves, for example in the <script> tag or in {#each ...} blocks

----> src/routes/+page.svelte

```
<script>
    let { message = `its greate to see you!` } = $props(); // Define a prop with a default value
</script>

<!-- Define the hello snippet -->
{#snippet hello(name)}
    <p>hello {name}! {message}!</p>
{/snippet}

<!-- To render a snippet, use a {@render ...} tag -->
{@render hello('svelte')}
{@render hello('Anjesh')}



| and they are 'visible' to everything in the same lexical scope (i.e. siblings,
| and children of those siblings):
|-------------------------------------------------------------------------
|    <div>
|        {#snippet x()}
|            {#snippet y()}...{/snippet}
|
|            <!-- this is fine -->
|           {@render y()}
|        {/snippet}
|
|        <!-- this will error, as `y` is not in scope -->
|        {@render y()}
|    </div>
|
|    <!-- this will also error, as `x` is not in scope -->
|    {@render x()}
|-------------------------------------------------------------------------------



| Snippets can reference themselves and each other
|-------------------------------------------------------------------------------
|    {#snippet blastoff()}
|         <span>🚀</span>
|    {/snippet}
|
|    {#snippet countdown(n)}
|        {#if n > 0}
|            <span>{n}...</span>
|            {@render countdown(n - 1)}
|    {:else}
|        {@render blastoff()}
|    {/if}
|    {/snippet}
|
|    {@render countdown(10)}
|
|-------------------------------------------------------------------------------
```

Tag : @html

To inject raw HTML into your component, use the {@html ...} tag:

-----> src/routes/+page.svelte

```
<!-- Define the HTML string -->
<script>
    let string = `here's some <strong>HTML!!!</strong>`;
</script>

<!-- Render the HTML string -->
<p>{@html string}</p>
```

Tag : @const

The @const directive allows you to declare a constant value that is computed from the current state of the component. It is useful for values that are derived from props or other reactive variables, and it ensures that the value is only computed once per render.

-----> src/routes/+page.svelte

```
<script>
    let boxes = [
        { width: 10, height: 10 },
        { width: 15, height: 30 }
    ];
</script>

<!-- The {@const ...} tag defines a local constant.  -->
{#each boxes as box, i (i)}
    {@const area = box.width * box.height}
    <p>
        {box.width} x {box.height} = <strong>{area}</strong>
    </p>
{/each}

Note
{@const} is only allowed as an immediate child of a block — {#if ...}, {#each ...}, {#snippet ...} and so on — 
a <Component /> or a <svelte:boundary>.
```

Tag : @debug

The {@debug ...} tag offers an alternative to console.log(...). It logs the values of specific variables whenever they change, and pauses code execution if you have devtools open.

-----> src/routes/+page.svelte

```
<script>
    let user = {
        firstname: 'John',
        lastname: 'Doe'
    };
</script>

<!-- Debug (console.log) user object -->
{@debug user}

<h1>Hello {user.firstname} {user.lastname}!</h1>

--------------------------------------------------------------------------------
<!- Compiles accepts a comma-separated list of variable names (not arbitrary expressions -->
{@debug user}
{@debug user1, user2, user3}

<!-- WON'T compile -->
{@debug user.firstname}
{@debug myArray[0]}
{@debug !isReady}
{@debug typeof user === 'object'}
```

What is : {@attach ...}

{@attach} is a Svelte 5 directive that gives your function direct access to the actual DOM node of any HTML element, so you can work on it directly.
It is the modern replacement of Svelte 4's use:action directive — cleaner, simpler, and more flexible.

What is Attachment Factory?
A regular attachment receives just the DOM node — nothing else. But sometimes you need to **pass extra values** to your attachment (like tooltip text, animation speed, config options).

So you write a **function that returns another function** — this is called an **Attachment Factory.**
An **Attachment Factory** is a function that accepts parameters and returns an attachment function,
used when your DOM logic needs extra data from outside.

Example of attach : Auto focus

Auto focus an input when the component mounts and the element is added to the DOM

-----> src/routes/+page.svelte

```
<script>
    function autoFocus(node) {
        node.focus(); // Focus on mount
    }
</script>

<!-- Focus on mount -->
<input type="text" {@attach autoFocus} placeholder="I get focus instantly" />
```

Example of Attachment Factory : Tool tip

Here example of tippy.js with Attachment Factory

-----> src/routes/+page.svelte

```
<script>
    import tippy from 'tippy.js';                       // Import the tippy.js library

    function tooltip(text) {                            // Function to attach the tooltip
        return (node) => {                              // Function to create the tooltip
            const t = tippy(node, { content: text });   // Create the tooltip
            return t.destroy;                           //Return a function to destroy the tooltip
        };
    }
</script>

<!-- Attach the tooltip pass the text as a parameter -->
<button {@attach tooltip('Click me')}>Hover me</button>
```

Example of Inline Attachment : Animation

Here a example of inline Attachment animate the Div

-----> src/routes/+page.svelte

```
<div
    {@attach (node) => {
        node.animate([{ opacity: 0 }, { opacity: 1 }], { duration: 1000 });
        // Animate the opacity from 0 to 1 over 1 second
    }}

➢ I fade in! </div>
```

Attach : Conditional attachment

Falsy values like false or undefined are treated as no attachment, enabling conditional usage:

-----> src/routes/+page.svelte

```svelte
<script>
    let isfocused = $state(true);

    // autofocus function
    function autofocus(node) {
        node.focus();
    }
</script>

<!- condationaly attach autofocus function to input -->
<input type="text" {@attach isfocused && autofocus} />

<!-- toggle focus -->
<button onclick={() => (isfocused = !isfocused)}>Toggel Focus</button>
```

Attach : Passing attachments to components

When used on a component, {@attach ...} will create a prop whose key is a Symbol. If the component then spreads props onto an element, the element will receive those attachments. This allows you to create wrapper components that augment elements

-----> src/routes/+page.svelte

```svelte
<script>
    import tippy from 'tippy.js';                          // Import tippy
    import Button from '$lib/components/Button.svelte';    // Import Button component
    let content = $state('Hello!');                        // Initialize content state

    // tooltip function
    function tooltip(content) {
        return (element) => {
            const tooltip = tippy(element, { content });
            return tooltip.destroy;
        };
    }
</script>

<!-- bind content -->
<input type="text" bind:value={content} />

<!-- attach tooltip function and pass content state value -->
<Button {@attach tooltip(content)}>Hover me</Button>
```

[-----> src/lib/components/Button.svelte]

```svelte
<script>
    let { children, ...props } = $props();
</script>

<!-- `props` includes attachments -->
<button {...props}>{@render children?.()}</button>
```

Bind : Function binding

You can also use bind:property={get, set}, where get and set are functions, allowing you to perform validation and transformation:

-----> src/routes/+page.svelte

```
<script>
    let value = $state('');
</script>

<input type="text" bind:value={
            () => value, //get function
            (v) => value = v.toLowerCase() //set function
 }/>
<p>You entered: {value}</p>
```

Bind : Input Value

Data ordinarily flows down, from parent to child. The bind: directive allows data to flow the other way, from child to parent.

-----> src/routes/+page.svelte

```
<script>
    let message = $state(''); //initial value of message is empty string
</script>

<!-- bind message to input value -->
<input type="text" bind:value={message} />

<!-- render message -->
<p>You entered: {message}</p>
```

Bind : Shorthand

bind:property={expression} links a property to a variable.If the variable name matches the property name, you can just write bind:value instead of bind:value={value}.

-----> src/routes/+page.svelte

```
<script>
    let value = $state('');
</script>

<!-- input bind:value -->
<input type="text" bind:value />

<p>You entered: {value}</p>
```

Bind : Input checkbox

Checkbox inputs can be bound with bind:checked:

-----> src/routes/+page.svelte

```
<script>
    let accepted = $state(false);
</script>

<!-- input bind:checked -->
<label>
    <input type="checkbox" bind:checked={accepted} />
    Accept terms and conditions
</label>

<!-- show accepted or not message conditionally -->
<p>{accepted ? 'Accepted' : 'Not accepted'}</p>
```

Bind : Checkbox bind:defaultChecked

Since 5.6.0, if an <input> has a defaultChecked attribute and is part of a form, it will revert to that value instead of false when the form is reset. Note that for the initial render the value of the binding takes precedence unless it is null or undefined.

----> src/routes/+page.svelte

```
<script>
    let checked = $state(true);
</script>

<!-- input bind:defaultChecked -->
<form>
    <input type="checkbox" bind:checked defaultChecked={true} />
    <input type="reset" value="Reset" />
</form>

<!-- show message when checkbox is checked or unchecked -->
<p>{checked ? 'Checked' : 'Not checked'}</p>
```

Bind : Checkbox bind:indeterminate check

The indeterminate property can be used to set the indeterminate state of a checkbox.

----> src/routes/+page.svelte

```
<script>
    let checked = $state(false); // checked is a boolean that reflects the state of the checkbox
    let indeterminate = $state(true); // indeterminate is a separate property from checked, so it
</script>                                                    needs to be bound separately

<!-- input bind:indeterminate  -->
<form>
    <input type="checkbox" bind:checked bind:indeterminate />

    <!-- show message based on checkbox state -->
    {#if indeterminate}
        waiting...
    {:else if checked}
        checked
    {:else}
        unchecked
    {/if}
</form>
```

Bind : Select bind:value

A <select> value binding corresponds to the value property on the selected <option>, which can be any value (not just strings, as is normally the case in the DOM).

----> src/routes/+page.svelte

```
<script>
    let selected = $state('');
</script>

<!-- select bind:value -->
<select bind:value={selected}>
    <option value="a">a</option>
    <option value="b">b</option>
    <option value="c">c</option>
</select>

<!-- show the selected value -->
<p>You selected: {selected}</p>
```

Bind : input bind:files

On <input> elements with type="file", you can use bind:files to get the FileList of selected files. When you want to update the files programmatically, you always need to use a FileList object. Currently FileList objects cannot be constructed directly, so you need to create a new DataTransfer object and get files from there.

DataTransfer is a special object that can be used to manipulate the files list of an input element. By creating a new DataTransfer object and assigning its files property to the files variable, we effectively clear the list of selected files.

----> src/routes/+page.svelte

```
<script>
    let files = $state();
    function clear() {
        files = new DataTransfer().files;
    }
</script>

<!-- input bind:files -->
<label for="avatar">Upload a picture : </label>
<input type="file" bind:files accept="image/png, image/jpeg" id="avatar" />

<!-- reset the files choosen (choose button) -->
<button onclick={clear}>clear</button>
```

Bind : input bind:group

Multiple inputs can be bound with bind:group, bind:group only works if the inputs are in the same Svelte component.

----> src/routes/+page.svelte

```
<script>
    let tortilla = $state('Plain');

    /** @type {string[]} */
    let fillings = $state([]); // Initialize fillings state
</script>

<h1>Customize your burrito</h1>
<!-- grouped radio inputs are mutually exclusive -->
<label><input type="radio" bind:group={tortilla} value="Plain" /> Plain</label>
<label><input type="radio" bind:group={tortilla} value="Whole wheat" /> Whole
wheat</label>
<label><input type="radio" bind:group={tortilla} value="Spinach" /> Spinach</label>

<!-- grouped checkbox inputs populate an array -->
<label><input type="checkbox" bind:group={fillings} value="Rice" /> Rice</label>
<label><input type="checkbox" bind:group={fillings} value="Beans" /> Beans</label>
<label><input type="checkbox" bind:group={fillings} value="Cheese" /> Cheese</label>
<label><input type="checkbox" bind:group={fillings} value="Guac (extra)" /> Guac
(extra)</label> -->

<p>Tortilla: {tortilla}</p>
<!-- display the selected fillings -->
<p>Fillings: {fillings.join(', ') || 'None'}</p>

<style>
    label {
        display: block;
    }
</style>
```

Bind : Audio

This example demonstrates how to use the <audio> element with Svelte's bind directive to create a simple audio player. It includes play/pause functionality, displays the current time and duration of the audio, and allows the user to seek through the audio using a range input.

-----> src/routes/+page.svelte

```
<script>
    // initialize state for duration, currentTime and paused using $state
    let src = 'https://actions.google.com/sounds/v1/alarms/digital_watch_alarm_long.ogg';
    let duration = $state(0);
    let currentTime = $state(0);
    let paused = $state(true);

    // helper function to format time in mm:ss
    function format(time) {
        const minutes = Math.floor(time / 60);
        const seconds = Math.floor(time % 60)
            .toString()
            .padStart(2, '0');
        return `${minutes}:${seconds}`;
    }
</script>

<!-- bind duration, currentTime and paused -->
<audio {src} bind:duration bind:currentTime bind:paused></audio>

<!-- toggle play/pause -->
<button onclick={() => (paused = !paused)}> {paused ? 'Play' : 'Pause'}</button>

<!-- format duration and currentTime -->
<p>{format(currentTime)} / {format(duration)}</p>

<!-- bind currentTime -->
<input type="range" min="0" max={duration} bind:value={currentTime} />
```

```
<audio> elements have their own set of bindings — five two-way and six read-only ones


two-way  | currentTime | playbackRate| paused   | volume  | muted |
-------------------------------------------------------------------------------------
readonly | duration    | buffered    | seekable | seeking | ended | readyState | played |
```

Bind : Image

<img> elements have two readonly bindings: naturalWidth, naturalHeight

-----> src/routes/+page.svelte

```
<script>
    // initialize state variables
    let src = $state('https://picsum.photos/800/600');
    let naturalWidth = $state(0);
    let naturalHeight = $state(0);
</script>

<!-- Render the image with bind naturalWidth and naturalHeight -->
<img {src} alt="sample" bind:naturalWidth bind:naturalHeight />

<!-- show image information image size and aspect ratio -->
<p>Natural Size: {naturalWidth} x {naturalHeight}</p>
<p>Aspect Ratio: {(naturalWidth / naturalHeight).toFixed(2)}</p>
```

Bind : Video

<video> elements have all the same bindings as <audio> elements, plus readonly videoWidth and videoHeight bindings.

-----> src/routes/+page.svelte

```
<script>
    // initialize state for duration, currentTime and paused using $state
    let src = 'https://sveltejs.github.io/assets/caminandes-llamigos.mp4';
    let poster = 'https://sveltejs.github.io/assets/caminandes-llamigos.jpg';
    let duration = $state(0);
    let currentTime = $state(0);
    let paused = $state(true);

    // helper function to format time in mm:ss
    function format(time) {
        const minutes = Math.floor(time / 60);
        const seconds = Math.floor(time % 60)
            .toString()
            .padStart(2, '0');
        return `${minutes}:${seconds}`;
    }
</script>

<!-- bind duration, currentTime and paused -->
<video {src} {poster} bind:duration bind:currentTime bind:paused>
    <track kind="captions" />
</video>

<!-- toggle play/pause -->
<button onclick={() => (paused = !paused)}> {paused ? 'Play' : 'Pause'}</button>

<!-- format duration and currentTime -->
<p>{format(currentTime)} / {format(duration)}</p>

<!-- bind currentTime -->
<input type="range" min="0" max={duration} bind:value={currentTime} />
```

Bind : bind:property for components

In this Example demonstrate tow way binding (child <----> parent) with a custom Input component

-----> src/routes/+page.svelte

```
<script>
    import Input from '$lib/components/Input.svelte';
    let value = $state(''); // Initialize value as an empty string
</script>

<!-- Render and bind the Input component -->
<Input bind:value />
<p>{value}</p>
```

```
<!-- src/lib/components/Input.svelte -->

<script>
    let { value = $bindable() } = $props();
</script>

<input type="text" bind:value />
```

Bind : details bind:open

<details> elements support binding to the open property.

----> src/routes/+page.svelte

```
<script>
    let isOpen = $state(false); // Initialize isOpen state
</script>

<!-- Toggle the isOpen state -->
<details bind:open={isOpen}>
    <summary>How do you comfort a JavaScript bug?</summary>
    <p>You console it.</p>
</details>

<!-- Render the isOpen state -->
<h1>{isOpen ? 'Open' : 'Closed'}</h1>
```

Bind : Contenteditable bindings

Elements with the contenteditable attribute support the following bindings: innerHTML innerText textContent

----> src/routes/+page.svelte

```
<script>
    // This example demonstrates the use of innerHTML, innerText, and textContent bindings in Svelte.
    let html = $state('<p>Edit <strong>me</strong>!</p>');
    let iText = $state('Hello innerText!');
    let tContent = $state('Hello textContent!');
</script>

<!-- innerHTML binding -->
<div contenteditable="true" bind:innerHTML={html}></div>
<pre>{html}</pre><hr />

<!-- innerText binding -->
<p contenteditable="true" bind:innerText={iText}></p>
<pre>{iText}</pre><hr />

<!-- textContent binding -->
<p contenteditable="true" bind:textContent={tContent}></p>
<pre>{tContent}</pre>
```

Bind : bind:this

In This Example we are using the bind:this directive to get a reference to the video element and then we can use that reference to play and pause the video.

----> src/routes/+page.svelte

```
<script>
    let src = 'https://sveltejs.github.io/assets/caminandes-llamigos.mp4';
    let myvideo = $state(); // myvideo is a reference to the video element
</script>

<!-- bind to myvideo -->
<video {src} bind:this={myvideo}>
    <track kind="captions" />
</video>

<!-- Play and Pause Myvideo -->
<button onclick={() => myvideo.play()}>play</button>
<button onclick={() => myvideo.pause()}>pause</button>
```

Bind : Dimensions

All visible elements have the following readonly bindings, measured with a ResizeObserver

----> src/routes/+page.svelte

```
<script>
    let clientWidth = $state(0);
    let clientHeight = $state(0);
    let offsetWidth = $state(0);
    let offsetHeight = $state(0);
    let contentRect = $state(null);
    let contentBoxSize = $state(null);
    let borderBoxSize = $state(null);
    let devicePixelContentBoxSize = $state(null);
</script>

<!-- Bind the element -->
<div
    class="box"
    bind:clientWidth
    bind:clientHeight
    bind:offsetWidth
    bind:offsetHeight
    bind:contentRect
    bind:contentBoxSize
    bind:borderBoxSize
    bind:devicePixelContentBoxSize
>
    Resize me!
</div>

<!-- Show the values -->
<p>clientWidth: {clientWidth} / clientHeight: {clientHeight}</p>
<p>offsetWidth: {offsetWidth} / offsetHeight: {offsetHeight}</p>
<p>contentRect: {contentRect?.width.toFixed(0)} x {contentRect?.height.toFixed(0)}</p>
<p>contentBoxSize: {contentBoxSize?.[0]?.inlineSize.toFixed(0)} x
{contentBoxSize?.[0]?.blockSize.toFixed(0)}</p>
<p>borderBoxSize: {borderBoxSize?.[0]?.inlineSize.toFixed(0)} x
{borderBoxSize?.[0]?.blockSize.toFixed(0)}</p>
<p>devicePixelContentBoxSize: {devicePixelContentBoxSize?.[0]?.inlineSize.toFixed(0)} x
{de1vicePixelContentBoxSize?.[0]?.blockSize.toFixed(0)}</p>

<!-- Style the element -->
<style>
    .box {width: 50%; padding: 20px; border: 5px solid steelblue;background: lightblue;
          resize: both; overflow: auto; min-height: 100px;}
</style>
```

```
+---------------------------+
|  margin                   |
|  +---------------------+  |
|  |  border             |  |
|  |  +---------------+  |  |
|  |  |  padding      |  |  |
|  |  |  +---------+  |  |  |
|  |  |  | content |  |  |  |
|  |  |  +---------+  |  |  |
|  |  +---------------+  |  |
|  +---------------------+  |
+---------------------------+

clientWidth  = content + padding
offsetWidth  = content + padding + border
contentRect  = ResizeObserver se (most accurate)
```

About : Await

As of Svelte 5.36, you can use the await keyword inside your components in three places where it was previously unavailable: at the top level of your component's <script>, inside $derived(...) declarations, inside your markup

This feature is currently experimental, and you must opt in by adding the experimental.async option wherever you configure Svelte, usually svelte.config.js:

Await : Enable It

-----> svelte.config.js

```
export default {
    compilerOptions: {
        experimental: {
            async: true // enable the experimental async feature
}}};
```

Await : Synchronized Updates

When awaited state changes, the UI waits for the async work to finish before updating — preventing inconsistent states.

-----> src/routes/+page.svelte

```
<script>
    let a = $state(1);
    let b = $state(2);
    //  helper async function to add a and b
    async function add() {
        await new Promise((f) => setTimeout(f, 1000)); // delay for 1 second
        return a + b;
    }
</script>

<!-- bind a and b -->
<input type="number" bind:value={a} />
<input type="number" bind:value={b} />

<!-- add a and b -->
<p>{a} + {b} = {await add(a, b)}</p>
```

Await : Error Handling

When an await expression throws an error, it bubbles up to the nearest <svelte:boundary> where the failed snippet handles it.

-----> src/routes/+page.svelte

```
<script>
    // thown error will be catched by failed block
    async function fetchData() {
        await new Promise((f) => setTimeout(f, 1000)); // simulate async operation
        throw new Error('Failed to fetch data'); // throw error
    }
</script>

<svelte:boundary>
     {#snippet pending()}                <!-- Render the pending blocks -->
        <p>Loading...</p>
    {/snippet}

    {#snippet failed(error)}            <!-- Render the failed block -->
        <p>Error : {error.message}</p>

    {/snippet}
        <p>{await fetchData()}</p>      <!-- Render the success block -->

</svelte:boundary>
```

Await Indicating loading states : pending snippet

1. pending snippet — first load onlyShown when the boundary first creates.
Disappears once async content resolves. Never shown again on subsequent updates.

-----> src/routes/+page.svelte

```
<script>
    // async function that simulates a delay and returns a string
    async function fetchData() {
        await new Promise((f) => setTimeout(f, 2000));
        return 'Hello from async!';
    }
</script>

<!-- Using await & svelte:boundary handel the promise state -->
<svelte:boundary>
        {#snippet pending()}        <!-- This block will be shown while the promise is pending -->

        <p>Loading... please wait</p>
    {/snippet}
       <p>{await fetchData()}</p>  <!-- This block will be shown when the promise resolves -->

</svelte:boundary>
```

Await Indicating loading states : $effect.pending()

$effect.pending() is a function that returns true when async work is in progress — for subsequent updates after the first load.
Used to show a spinner or loading indicator.

-----> src/routes/+page.svelte

```
<script>
    let username = $state('');

    // simulate an async validation function
    async function validate(value) {
        await new Promise((f) => setTimeout(f, 1000));
        return value.length > 3 ? 'valid' : 'invalid';
    }
</script>

<!-- bind username -->
<input type="text" bind:value={username} placeholder="Enter your username" />

<svelte:boundary> <!-- check promises resolved or pending with $effect.pending -->


    {#if $effect.pending()} <!-- check if a promise is pending -->

        <p>checking...</p>
    {:else}

  <span>{await validate(username)}</span>  <!-- show validation result when promise is resolved -->

    {/if}
</svelte:boundary>
```

Await Indicating loading states : tick() and settled()

tick() flushes a single state change to the UI immediately, before it gets grouped with other changes. settled() waits until all async DOM updates are fully applied.

----> src/routes/+page.svelte

```
<script>
    import { tick, settled } from 'svelte'; // import tick and settled functions from svelte

    let color = $state('red');              // initialize state
    let answer = $state(-1);
    let updating = $state(false);

    async function onclick() {
        updating = true;

        // updating state values and waiting for the DOM to update
        await tick();
        await new Promise((f) => setTimeout(f, 1000));
        color = 'octarine';
        answer = 42;

        // wait for all pending state updates to be processed
        await settled();

        updating = false;
    }
</script>

<!-- showing values -->
<p>Color: <strong>{color}</strong></p>
<p>Answer: <strong>{answer}</strong></p>
<p>Status: <strong>{updating ? 'Updating...' : 'Done'}</strong></p>

<button {onclick}>Update</button>
```

Await : Concurrency

Independent await expressions in markup run in parallel automatically. However, sequential await inside <script> or async functions Runes one after another like normal JavaScript. Independent $derived expressions update independently after their first creation, even if created sequentially. Markup — Runes in parallel

----> src/routes/+page.svelte

```
<script>
    let x = $state(1);
    let y = $state(2);

  // helper functions that run asynchronously, but we want them to run at the same time, not one after another
    async function one(x) {
        await new Promise((f) => setTimeout(f, 1000));
        return x * 10;
    }

    async function two(y) {
        await new Promise((f) => setTimeout(f, 500));
        return y * 20;
    }
</script>

<!-- both run at the same time, not one after another -->
<p>one: {await one(x)}</p>
<p>two: {await two(y)}</p>
```

Await : Forking — Preload Data Before Click

fork() starts async work in the background when user hovers. When user clicks, data is already loaded — no waiting. commit() applies the work, discard() cancels it.

-----> src/routes/+page.svelte

```
<script>
  import { fork } from 'svelte'; // import the fork function from svelte
  import Data from '$lib/components/Data.svelte'; // import the Data component

  let open = $state(false);          // create a reactive state variable 'open' initialized to false
  let pending = null;                // variable to hold the pending work

  function preload() {                          // function to preload data on hover
    pending ??= fork(() => (open = true));   // start async work in background on hover
  }

  function discard() {                          // function to discard pending work if user moves away
    pending?.discard();                         // cancel if user moves away
    pending = null;
  }
</script>

<button                                          <!--  button to preload data on hover -->
  onpointerenter={preload}                       // preload data on hover
  onpointerleave={discard}                       // discard pending work if user moves away
  onclick={() => {
    pending?.commit();                           // apply preloaded work on click
    pending = null;
    open = true;                                 // fall back if pending did not exist
  }}>Hover then click</button>

<!-- show Data  component when open is true -->
{#if open}
  <Data onclose={() => (open = false)} />
{/if}
```

-----> src/lib/components/Data.svelte

```
<script>
    let { onclose } = $props(); // receive onclose from parent component

        async function fetchData() {                          // simulate async data fetching
        await new Promise((f) => setTimeout(f, 2000));        // simulate delay
        return 'Data loaded successfully!';
    }
</script>

<!-- boundary handles loading and error states -->
<svelte:boundary>
    {#snippet pending()}                        <!-- show while fetchData is resolving -->
        <p>Loading data...</p>
    {/snippet}

    {#snippet failed(error)}                    <!-- show if fetchData throws an error  -->
        <p>Error: {error.message}</p>
    {/snippet}

     <div>                                      <!-- show  when fetchData resolves successfully -->
        <p>{await fetchData()}</p>
        <button onclick={onclose}> Close</button>
    </div>
</svelte:boundary>
```

Transition : Example

A transition animates elements entering or leaving the DOM on state change. All elements in a block stay in the DOM until every transition completes. It's bidirectional — meaning it can smoothly reverse if interrupted mid-animation.

----> src/routes/+page.svelte

```
<script>
    import { fade } from 'svelte/transition'; // import the fade transition
    let visible = $state(false); // Initialize a state variable to control visibility
</script>

<!-- toggle button -->
<button onclick={() => (visible = !visible)}> Toggle </button>

<!-- use fade transition -->
{#if visible}
    <div transition:fade>Hello world</div>
{/if}
```

Transition : Transition parameters

Transitions can have parameters.(The double {{curlies}} aren't a special syntax; this is an object literal inside an expression tag.)

----> src/routes/+page.svelte

```
<script>
    import { fade } from 'svelte/transition'; // import Built-in transitions fade
    let visible = $state(true);               // initially visible
</script>

<!-- toggle visibility -->
<button onclick={() => (visible = !visible)}>Toggle Visibility</button>

<!-- fade in and out -->
{#if visible}
    <div transition:fade={{ duration: 200 }}>fade in and out over tow seconds</div>
{/if}
```

Transition : Transition events

An element with transitions will dispatch the following events in addition to any standard DOM events:
Introstart, introend, outrostart, outroend

----> src/routes/+page.svelte

```
<script>
    import { fly } from 'svelte/transition'; // import Built-in transitions fly
    let visible = $state(true);
    let status = $state('');
</script>

<!-- toggle visibility and show status -->
<button onclick={() => (visible = !visible)}> {visible ? 'Hide' : 'Show'}</button>
<p>{status}</p>

{#if visible}
    <p transition:fly={{ y: 200, duration: 2000 }}      // fly animation
        onintrostart={() => (status = 'Intro started')} // event handler for intro start
        onoutrostart={() => (status = 'Outro started')} // event handler for outro start
        onintroend={() => (status = 'Intro ended')}     // event handler for intro end
        onoutroend={() => (status = 'Outro ended')}     // event handler for outro end
    > Files in and out</p>
{/if}
```

Transition : Local and global

Transitions are local by default. Local transitions only play when the block they belong to is created or destroyed, not when parent blocks are created or destroyed.
Local transition — only animates when its own block is created/destroyed.
Global transition — animates when any parent block is created/destroyed.

----> src/routes/+page.svelte

```
<script>
    import { fade } from 'svelte/transition'; // import the fade transition

    let x = $state(true);
    let y = $state(true);
</script>

<!-- toggle x and y -->
<button onclick={() => (x = !x)}> Toggle x</button>
<button onclick={() => (y = !y)}> Toggle y</button>

<!-- show x and y -->
<p>x: {x}, y: {y}</p>

<!-- animate on x or y toggle -->
{#if x}
    {#if y}
        <p transition:fade>Local - animate on Y toggle</p>
        <p transition:fade|global>Galobal Animate on X or Y toggle</p>
    {/if}
{/if}
```

Transition : Custom transition function

here example of a custom transition function that scales an element from 0 to its original size using the elasticOut easing function for a bouncy effect the whoosh function takes a node and parameters for delay, duration, and easing, and returns an object that defines the transition behavior

----> src/routes/+page.svelte

```
<script>
    import { elasticOut } from 'svelte/easing'; // import elasticOut easing function
    let visible = $state(false);

   // custom transition function that scales the element from 0 to its original size
    function whoosh(node, params) {
        const existingTransform = getComputedStyle(node).transform.replace('none', '');
        return {
            delay: params.delay || 0, // default delay of 0
            duration: params.duration || 400, //  default duration of 400ms
            easing: params.easing || elasticOut, // use elasticOut easing for a bouncy effect

            // apply existing transform and scale based on progress (t)
            css: (t, u) => `transform: ${existingTransform} scale(${t})`
        };
    }
</script>

<!-- toggle visibility -->
<button onclick={() => (visible = !visible)}>toggle visibility</button>

<!-- animate -->
{#if visible}
    <div in:whoosh>wooshes in</div>
{/if}
```

Transition : in: and out:

The in: and out: directives are identical to transition:, except that the resulting transitions are not bidirectional — an in transition will continue to 'play' alongside the out transition, rather than reversing, if the block is outroed while the transition is in progress. If an out transition is aborted, transitions will restart from scratch.

-----> src/routes/+page.svelte

```
<script>
    import { fade, fly } from 'svelte/transition'; // import fade and fly
    let visible = $state(false);
</script>

<!-- toggle visibility -->
<label>
    <input type="checkbox" bind:checked={visible} /> visible
</label>

<!-- animate in and out -->
{#if visible}
    <div in:fly={{ y: 200 }} out:fade>flies in,fades out</div>
{/if}
```

Svelte : Built-in transitions

A selection of built-in transitions can be imported from the svelte/transition module.

-----> src/routes/+page.svelte

```
<script>
   // import the transitions
    import { fade, fly, scale, slide, blur, draw } from 'svelte/transition';
    let visible = $state(false);
</script>

<!-- button to toggle -->
<button onclick={() => (visible = !visible)}>Toggle</button>

{#if visible}
    <!-- transitions : fade, fly, scale, slide, blur -->
    <p transition:fade>Fade</p>
    <p transition:fly={{ y: 50 }}>Fly</p>
    <p transition:scale>Scale</p>
    <p transition:slide>Slide</p>
    <p transition:blur>Blur</p>

    <!-- draw transition (draws the path) -->
    <svg viewBox="0 0 124 124" width="200">
        <path transition:draw d="M 0 25 L 100 25" fill="none" stroke="red" stroke-width="2" />
    </svg>
{/if}
```

Animation : Flip

An animation is triggered when the contents of a keyed each block are re-ordered. Animations do not run when an element is added or removed, only when the index of an existing data item within the each block changes. Animate directives must be on an element that is an immediate child of a keyed each block.
Animations can be used with Svelte's built-in animation functions or custom animation functions.

-----> src/routes/+page.svelte

```
<script>
    import { flip } from 'svelte/animate'; // import flip animation
    let list = $state(['Apple', 'Banana', 'Mango', 'Orange', 'Watermelon']);

    // shuffle the list
    function shuffle() {
        list = list.sort(() => Math.random() - 0.5);
    }
</script>

<!-- shuffle -->
<button onclick={shuffle}> Shuffle</button>

<!-- animate:flip -->
{#each list as item (item)}
    <div animate:flip={{ duration: 500 }}>{item}</div>
{/each}
```

Animation : Custom animation functions

Custom animation functions receive three arguments:node — the DOM element{ from, to } — element's start and end positions as DOMRects (x, y, width, height)params — any extra parameters passed to the animationThe css function Runes repeatedly before animation starts, receiving t (0→1) and u (1-t). Svelte uses the returned CSS to create a web animation on the element.

-----> src/routes/+page.svelte

```
<script>
    import { cubicOut } from 'svelte/easing'; // Import the easing

    let list = $state(['Apple', 'Banana', 'Mango', 'Orange']);


    // Define the animation
    function whizz(node, { from, to }, params) {
        const dx = from.left - to.left; // Calculate the horizontal distance
        const dy = from.top - to.top; // Calculate the vertical distance
        const d = Math.sqrt(dx * dx + dy * dy); // Calculate the diagonal distance

        return {
            delay: 0, // No delay
            duration: Math.sqrt(d) * 120, // Duration based on distance
            easing: cubicOut, // Use cubicOut easing
            // The CSS transform to apply during the animation
            css: (t, u) => `transform: translate(${u * dx}px, ${u * dy}px) rotate(${t * 360}deg)`
        };
    }
</script>

<!-- Shuffle button -->
<button onclick={() => (list = [...list].sort(() => Math.random() - 0.5))}>Shuffle</button>

<!-- Render the list -->
{#each list as item (item)}
    <div animate:whizz>{item}</div>
{/each}
```

About : Class

There are two ways to set classes on elements: the class attribute and the class: directive.
class Attribute (Preferred)Accepts primitives, objects, and arrays. Objects apply truthy keys as classes.
Arrays skip falsy values. Uses clsx internally. Composes cleanly across components. Available since Svelte 5.16.

ClassValue TypeSince Svelte 5.19, import ClassValue from svelte/elements for type-safe class props in TypeScript.

class: Directive (Legacy)Conditionally applies a single class. Supports shorthand when variable name matches class name. Still valid but less composable than the attribute form. Avoid in new code.

Class : Attributes

Primitive values work like any standard HTML attribute — pass a plain string or JavaScript expression directly to class. Falsy values like false or NaN are stringified, while null and undefined omit the attribute entirely.

-----> src/routes/+page.svelte

```
<script>
    let large = $state(false); // Initialize large state
</script>

<!-- toggle between large and small -->
<p class={large ? 'large' : 'small'}>Hello Svelte!</p>

<!-- toggle -->
<button onclick={() => (large = !large)}>Toggle</button>

<!-- styles -->
<style>
    .large { font-size: 32px; }
    .small { font-size: 16px; }
</style>
```

Class : Objects

Object truthy keys are applied as classes. Falsy keys are omitted entirely.

-----> src/routes/+page.svelte

```
<script>
    let { cool } = $props(); // `cool` is a prop passed to this component
</script>

<!-- results in `class="cool"` if `cool` is truthy, `class="lame"` otherwise -->
<p class={{ cool, lame: !cool }}>Hello Svelte!</p>

<!-- style -->
<style>
    .cool { color: green; }
    .lame { color: red;   }
</style>
```

Class : The class: directive

class: was the old way to conditionally apply classes before Svelte 5.16. Prefer class attribute in new code.

-----> src/routes/+page.svelte

```
<!-- These are equivalent -->
<div class={{ cool, lame: !cool }}>...</div>
<div class:cool={cool} class:lame={!cool}>...</div>

<!-- shorthand – variable name matches class name -->
<div class:cool class:lame={!cool}>...</div>
```

Class : Arrays

Array truthy values are combined into a single class string. Falsy values are skipped.

-----> src/routes/+page.svelte

```
<script>
    let faded = $state(false);
    let large = $state(false);
</script>


<!-- if `faded` and `large` are both truthy, results in `faded large` style applied -->
<div class={[faded && 'faded', large && 'large']}></div>


<!-- toggle faded and large -->
<button onclick={() => (faded = !faded)}>Toggle faded</button>
<button onclick={() => (large = !large)}>Toggle large</button>


<!-- style -->
<style>
    div { width: 100px; height: 100px; background-color: red; }
    .faded { opacity: 0.5; }
    .large { width: 200px; height: 200px; }
</style>
```

Class : Merge Classes to Components

Pass classes from a parent into a child component. The child merges them with its own base classes using an array

-----> src/routes/+page.svelte

```
<script>
    import Button from '$lib/components/Button.svelte'; // Import the Button component
    let useStyle = $state(false); // Create a reactive state variable to toggle the style
</script>


<!-- Toggle the style -->
<Button onclick={() => (useStyle = !useStyle)} class={{ 'my-custom-class': useStyle }}>Toggle Style</Button>


<style>
    /* Apply a style to all elements with the class "my-custom-class" */
    :global(.my-custom-class) { background-color: red; width: 50%;}
</style>
```

-----> src/lib/components/Button.svelte

```
<script>
    let props = $props(); // Get the props passed to the Button component
</script>


<!-- Render the button with custom classes and children -->
<button {...props} class={['cool-button', props.class]}> {@render props.children?.()}
</button>
```

Style : Basic

The style: directive provides a shorthand for setting multiple styles on an element.

-----> src/routes/+page.svelte

```
<!-- These are equivalent -->
<div style:color="red">Hello, Svelte Developer</div>
<div style="color: red">Hello, Svelte Developer</div>
```

Style : Shorthand

The shorthand form is allowed only if variable & property name same (name = property name)

-----> src/routes/+page.svelte

```
<script>
    let color = $state('red'); // Initialize color state
</script>

<div style:color>Hello Svelte Developer</div> <!-- Short hand syntax -->
```

Style : JS expression

The value can contain arbitrary expressions:

-----> src/routes/+page.svelte

```
<script>
    let Mycolor = $state('red'); // Initialize color state
</script>

<div style:color={Mycolor}>Hello Svelte Developer</div> <!-- Short hand syntax -->
<input type="color" bind:value={Mycolor} />
```

Style : Multiple styles

Multiple styles can be set on a single element:

-----> src/routes/+page.svelte

```
<script>
    let color = $state('red'); // Reactive state for color
    let darkMode = $state(false); // Reactive state for dark mode
</script>

<!-- Multiple style properties -->
<div style:color style:width="12rem" style:background-color={darkMode ? 'black' : 'white'}> Hello Svelte!
</div>

<!-- Bind color & toggle dark mode -->
<input type="color" bind:value={color} />
<button onclick={() => (darkMode = !darkMode)}> {darkMode ? 'Light Mode' : 'Dark Mode'}</button>
```

Style : Important

To mark a style as important, use the |important modifier:

-----> src/routes/+page.svelte

```
<script>
    let color = $state('red'); // Reactive state for color
</script>

<!-- Important style -->
<div style:color|important="blue">Hello Svelte!</div>

<!-- Bind color -->
<input type="color" bind:value={color} />
```

Style : Precedence

In Svelte, style: directive always has the highest precedence over the style attribute.

----> src/routes/+page.svelte

```
<!-- showing styles higher priority -->

<div style:color="red" style="color: blue">This will be read</div>
<div style:color="red" style="color: blue !important">This will still be read</div>
```

Style : Custom properties

CSS custom properties let you control CSS values from JavaScript dynamically.

----> src/routes/+page.svelte

```
<script>
    let primaryColor = $state(''); // Initialize state for primary color
    let fontSize = $state(''); // Initialize state for font size
</script>

<!-- dynamic styles -->
<div style:--primary={primaryColor} style:--size={fontSize + 'px'} class="box">Hello
Svelte!</div>

<!-- bind primaryColor and fontSize -->
<input type="color" bind:value={primaryColor} placeholder="Primary Color" />
<input type="range" min="12" max="48" bind:value={fontSize} placeholder="Font Size" />

<!-- dynamic styles -->
<style>
    .box {
        color: var(--primary);
        font-size: var(--size);
    }
</style>
```

Style Scope : Scoped keyframes

@keyframes defined inside a component are scoped to that component only — so two components can have the same animation name without conflicting.

----> src/routes/+page.svelte

```
<script>
    let bouncing = $state(false);
</script>

<!-- toggle bouncing -->
<button onclick={() => (bouncing = !bouncing)}>Toggle</button>

<!-- animate when bouncing is true -->
<div class:bouncy={bouncing}>Bounce!</div>

<style>
    .bouncy {
        animation: bounce 0.5s infinite;
    }
   /* bounce animation */
    @keyframes bounce {
        0%, 100% { transform: translateY(0);}
        50% { transform: translateY(-30px);}
    }
</style>
```

Style Scope : Scoped styles

CSS inside <style> is scoped to that component only — it won't affect other components. Svelte does this by adding a unique hashed class (e.g. svelte-123xyz) to every affected element automatically.

----> src/routes/+page.svelte

```
<p>Hello Svelte!</p>

<style>
    p {
      color: red; /* this will only affect <p> elements in this component */
    }
</style>
```

Global Style : :global(...)

To apply styles to a single selector globally, use the :global(...) modifier:

----> src/routes/+page.svelte

```
<!-- applies to all <p> elements -->
<p class="big red">hello Svelte!</p>

<!-- applies to all <strong> elements -->
<div><strong>Hello Strong Svelte!</strong></div>

<style>
    :global(body) {
        margin: 0; /* applies to <body> */
    }
    div :global(strong) {
        /* applies to all <strong> elements, in any component that are inside <div> elements belonging to
        color: goldenrod;                                                                this component */

    }
    p:global(.big.red) {
        /* applies to all <p> elements belonging to this component with `class="big red"`, even if it is
        color: red;                          applied programmatically (for example by a library) */
        font-size: 3rem;
    }
</style>
```

Global Style : @keyframes

To make @keyframes globally accessible,
prefix the name with -global-. Svelte removes this prefix at compile time, so you reference it without -global- everywhere else.

Define → @keyframes -global-fadebound (with -global-)
Use → animation: fadebounce (without -global-) -->

----> src/routes/+page.svelte

```
<div class="box">Animated Box</div>

<style>
    /* Use Animation without -global- prefix */
    .box {  animation: fadebounce 0.5s ease-in-out; }

    /* defined with -global- prefix */
    @keyframes -global-fadebounce {
        0% { opacity: 0; transform: translateY(-20px);}
        100% { opacity: 1; transform: translateY(0);}
    }
</style>
```

Global Style : :global

To apply styles to a group of selectors globally, create a :global {...} block:
.a :global { .b .c .d } can also be written as .a :global .b .c .d — everything after :global is unscoped. Both are equivalent, but the nested form is preferred.

-----> src/routes/+page.svelte

```
<!-- Divs and Paragraphs -->
<div>Global Div</div>
<p>Global Paragraph</p>

<!-- Nested elements -->
<div class="a">
    <div class="b">
        <div class="c">
            <div class="d">Nested Target</div>
        </div>
    </div>
</div>

<style>
    /* Global styles */
    :global {
        div { background: lightblue; margin: 5px; padding: 5px; }
        p {   color: red; font-size: 1.5rem; }
    }

    /* Nested selectors select all elements inside the target */
    .a :global {
        .b .c .d { background: goldenrod; color: white; }
    }
</style>
```

Style : Custom properties

CSS Custom Properties on Components — pass dynamic styles from parent to child component using --property-name syntax.

-----> src/routes/+page.svelte

```
<script>
    import Box from '$lib/components/Box.svelte';
    let color = $state('');
</script>

<!-- Passing a custom property directly -->
<Box --box-color={color} />

<!-- changing the color -->
<input type="color" bind:value={color} placeholder="Enter a color" />
```

```
<!-- file: src/lib/components/Box.svelte -->

<!-- Using a custom property -->
<div class="box"></div>

<!-- Using a custom property -->
<style>
    .box {
        /* Use the variable with a fallback value */
        background-color: var(--box-color, #ccc);
        width: 100px; height: 100px;
    }
</style>
```

Style : Nested <style> elements

Only one top-level <style> allowed per component. Nested <style> inside elements is possible,
but it's inserted into the DOM as-is — no scoping, no processing — so it affects every matching element in the entire app.

----> src/routes/+page.svelte

```
<script>
    // Importing components from the components directory
    import Component1 from '$lib/components/Component1.svelte';
    import Component2 from '$lib/components/Component2.svelte';
</script>

<!-- Rendering the components -->
<Component1 />
<Component2 />

<!-- This paragraph is also red -->
<p>This paragraph is also red!</p>
```

```
<!-- Component1.svelte -->

<div class="component1">
    <p>Component 1 text</p>

    <!-- nested style — affects ALL divs in entire app -->
    <style>
        p {
            color: red;
            font-size: 2rem;
        }
    </style>
</div>
```

```
<!-- Component2.svelte -->

<div class="component2">
    <p>Component 2 text — also red!</p>
</div>
```

About : <svelte:boundary>

Boundaries allow you to 'wall off' parts of your app, so that you can:

provide UI that should be shown when await expressions are first resolving
handle errors that occur during rendering or while running effects, and provide UI that should be rendered when an error happens
If a boundary handles an error (with a failed snippet or onerror handler, or both) its existing content will be removed.

Errors occurring outside the rendering process (for example, in event handlers or after a setTimeout or async work) are not caught by error boundaries.
Properties For the boundary to do anything, one or more of the following must be provided.

Svelte: boundary : pending

This snippet will be shown when the boundary is first created, and will remain visible until all the await expressions inside the boundary have resolved

-----> src/routes/+page.svelte

```
<script>
    //  async function that simulates fetching data with a delay
     async function fetchData(value) {
         return new Promise((resolve) => { setTimeout(() => resolve(value) }, 1000);});
     }
</script>

<!-- boundary handles pending (loading) states -->
<svelte:boundary>

     <!-- show  when fetchData resolves successfully -->
     <p>{await fetchData('Hello svelte!')}</p>

    <!-- show while fetchData is resolving -->
     {#snippet pending()}
         <p>loading...</p>
     {/snippet}
</svelte:boundary>
```

Svelte: boundary : failed

When an error is thrown inside the boundary, the failed snippet is shown with two arguments — error (the error object) and reset (function to recreate the content and try again).

-----> src/routes/+page.svelte

```
<script>
     async function fetchData() {
         await new Promise((f) => setTimeout(f, 1000)); // simulate a delay
         throw new Error('Failed to fetch data'); // throw an error
     }
</script>

<!-- boundary handles loading and error states of await fetchData -->
<svelte:boundary>
     {#snippet pending()}                  <!-- show while fetchData is resolving pending state -->
         <p>Loading data...</p>
     {/snippet}

     {#snippet failed(error, reset)}       <!-- show if fetchData throws an error -->
         <p>Error: {error.message}</p>
         <button onclick={reset}>Try again</button>     <!-- reset button for retry -->
     {/snippet}

         <p>{await fetchData()}</p>        <!-- show when fetchData resolves successfully -->
</svelte:boundary>
```

Svelte: boundary : onerror

Same as failed but error UI is shown outside the boundary. failed keeps error UI inside, onerror lets you place it anywhere in the component. If an error occurs inside onerror itself, it bubbles up to the nearest parent boundary.

Warning:
<svelte:boundary> does not catch SSR errors by default — if an error occurs during SSR, the entire page render crashes and onerror is never called. For client-side boundary testing in SvelteKit, disable SSR: create file name “+page.js”
Add this code “export const ssr = false;”

-----> src/routes/+page.svelte

```
<script>
    import FlakyComponent from '$lib/components/FlakyComponent.svelte';// import a component

    let error = $state(null);
    let reset = $state(() => {});

    // onerror callback receives the error and a reset function that can be called to reset the boundary
    function onerror(e, r) {
        error = e;
        reset = r;
    }
</script>

<!-- boundary has no error UI inside -->
<svelte:boundary {onerror}><FlakyComponent /></svelte:boundary>

<!-- error UI is outside boundary -->
{#if error}
    <p>Error: {error.message}</p>
    <button
        onclick={() => {
            error = null;                          // clear the error
            reset();                               // call reset to reset the boundary and try again
        }}>Try again</button
    >
{/if}
```

-----> src/lib/components/FlakyComponent.svelte

```
<script>

   //  async operation that simulates a failure
    async function fetchData() {
        await new Promise((f) => setTimeout(f, 1000));
        throw new Error('Something went wrong!');
    }

    // rethrow error to be caught by the boundary
    async function loadData() {
        try {
            return await fetchData();
        } catch (e) {
            console.error('Error in FlakyComponent:', e);
            throw e; // rethrow to be caught by the boundary
        }
    }
</script>

<p>{await loadData()}</p>
```

About : <svelte:window>

The <svelte:window> element allows you to add event listeners to the window object without worrying about removing them when the component is destroyed, or checking for the existence of window when server-side rendering.

This element may only appear at the top level of your component — it cannot be inside a block or element

```
<svelte:window onevent={handler} />
<svelte:window bind:prop={value} />
```

Svelte: window : Event listener

Adds event listeners to the window object without worrying about removing them when the component is destroyed. Must be at the top level of your component — not inside a block or element.

-----> src/routes/+page.svelte

```
<script>
    // event handler
    function handleEvent(event) {
        alert(`Press the ${event.key} key!`);
    }
</script>

<!-- window event listener (onkeydown) -->
<svelte:window onkeydown={handleEvent} />
```

Svelte: window : Bindable Properties

Bind to window properties to reactively track browser dimensions, scroll position, and network status. All properties are readonly except scrollX and scrollY which can be written to trigger scrolling.

-----> src/routes/+page.svelte

```
<script>
    let innerWidth = $state(0);                 // initialize innerWidth state variable
    let innerHeight = $state(0);                // initialize innerHeight state variable
    let scrollY = $state(0);                    // initialize scrollY state variable
</script>

<!-- bind innerWidth, innerHeight and scrollY -->
<svelte:window bind:innerWidth bind:innerHeight bind:scrollY />

<!-- display innerWidth, innerHeight and scrollY -->
<p>Inner width: {innerWidth}</p>
<p>Inner height: {innerHeight}</p>
<p>Scroll Y: {scrollY}</p>
```

Svelte: window : Available bindable properties:

Property	Type
innerWidth	Readonly
innerHeight	Readonly
outerWidth	Readonly
outerHeight	Readonly
scrollX	Writable
scrollY	Writable
online	Readonly
devicePixelRatio	Readonly

About : <svelte:document>

Similarly to <svelte:window>, this element allows you to add listeners to events on document, such as visibilitychange, which don't fire on window. It also lets you use attachments on document.
As with <svelte:window>, this element may only appear the top level of your component and must never be inside a block or element.

```
<svelte:document onevent={handler} /> <svelte:document bind:prop={value} />
```

Svelte: document : Event listener

Listens for visibilitychange on the document — fires when the user switches tabs or minimizes the window. Returns "visible" when active and "hidden" when inactive.

-----> src/routes/+page.svelte

```
<script>
    // Define a function to handle the visibilitychange event
    function handlevisivilitychange() {
        console.log(document.visibilityState);
    }
</script>

<!-- Add an event listener to the document -->
<svelte:document onvisibilitychange={handlevisivilitychange} />
```

Svelte: document: Bindable Properties

<svelte:document> — Bind visibilityStateBind visibilityState to reactively track whether the page is visible or hidden. Returns "visible" when active and "hidden" when tab is switched or window is minimized.

-----> src/routes/+page.svelte

```
<script>
    let visibilityState = $state(''); // Initialize visibilityState
</script>

<!-- bind visibilityState (shorthand) -->
<svelte:document bind:visibilityState />

<!-- display visibilityState -->
<p>Visiblity State: {visibilityState}</p>
```

Svelte: window : Available bindable properties:

Property	Type
activeElement	Readonly
fullscreenElement	Readonly
pointerLockElement	Readonly
visibilityState	Readonly

Svelte: window : Specific Events

Event	Fires when
Visibilitychange	Tab switch or window minimize
Fullscreenchange	Enter or exit fullscreen
Selectionchange	Text selection changes
Pointerlockchange	Pointer lock changes

Special elment : <svelte:body>

Similarly to <svelte:window>, this element allows you to add listeners to events on document.body, such as mouseenter and mouseleave, which don't fire on window. It also lets you use actions on the <body> element.
As with <svelte:window> and <svelte:document>, this element may only appear at the top level of your component and must never be inside a block or element.

-----> src/routes/+page.svelte

```
<script>
    // handle mouse enter and leave
    function handleMouseEnter() {
        console.log('mouse enter');
    }

    function handleMouseLeave() {
        console.log('mouse leave');
    }
</script>

<!-- call handleMouseEnter and handleMouseLeave on mouse enter and leave events -->
<svelte:body onmouseenter={handleMouseEnter} onmouseleave={handleMouseLeave} />

<!-- set min-height to 100vh and remove margin -->
<style> :global(body) { min-height: 100vh; margin: 0; } </style>
```

Special elment : <svelte:head>

This element makes it possible to insert elements into document.head. During server-side rendering, head content is exposed separately to the main body content.
As with <svelte:window>, <svelte:document> and <svelte:body>, this element may only appear at the top level of your component and must never be inside a block or element.
Inserts elements into document.head — useful for setting page title, meta tags, and SEO. Must be at top level of your component.

-----> src/routes/+page.svelte

```
<!-- Insert the title and meta description using svelte:head -->
<svelte:head>
    <title>Home</title>
    <meta name="description" content="This is where the description goes for SEO" />
</svelte:head>
```

Meta Tags : for SEO

```
<svelte:head>
    <!-- SEO -->
    <meta name="description" content="Page description" />
    <meta name="keywords" content="svelte, javascript, web" />
    <meta name="author" content="Your Name" />
    <meta name="robots" content="index, follow" />

    <!-- Viewport -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <!-- Open Graph (Social Media) -->
    <meta property="og:title" content="Page Title" />
    <meta property="og:description" content="Page description" />
    <meta property="og:image" content="https://example.com/image.jpg" />
    <meta property="og:url" content="https://example.com" />
    <meta property="og:type" content="website" />

    <!-- Canonical URL -->
    <link rel="canonical" href="https://example.com" />

    <link rel="icon" href="/favicon.ico" /> <!-- Favicon -->
    <meta charset="UTF-8" /> <!-- Charset -->
</svelte:head>
```

Special elment : <svelte:element>

The <svelte:element> element lets you render an element that is unknown at author time, for example because it comes from a CMS. Any properties and event listeners present will be applied to the element.

The only supported binding is bind:this, since Svelte's built-in bindings do not work with generic elements.

If this has a nullish value, the element and its children will not be rendered.

If this is the name of a void element (e.g., br) and <svelte:element> has child elements, a runtime error will be thrown in development mode:

-----> src/routes/+page.svelte

```
<script>
    let tag = $state('h1');
</script>

<!-- Only bind:this is supported — no other bindings
Void elements like br, hr, img cannot have children Use xmlns for SVG elements -->

<svelte:element this={tag}> This is dynamic tag</svelte:element>

<!-- SVG element -->
<svelte:element this={tag} xmlns="http://www.w3.org/2000/svg" />
```

Special elment : <svelte:options>

The <svelte:options> element provides a place to specify per-component compiler The possible options are:

runes={true} — forces a component into runes mode
runes={false} — forces a component into legacy mode
namespace="..." — the namespace where this component will be used, can be "html" (the default), "svg" or "mathml"
customElement={...} — the options to use when compiling this component as a custom element. If a string is passed,
css="injected" — the component will inject its styles inline: During server-side rendering, it's injected as a <style> tag in the head, during client side rendering, it's loaded via JavaScript

-----> src/routes/+page.svelte

```
<!-- force runes mode -->
<svelte:options runes={true} />

<!-- force legacy mode -->
<svelte:options runes={false} />

<!-- svg namespace -->
<svelte:options namespace="svg" />

<!-- custom element -->
<svelte:options customElement="my-custom-element" />

<!-- inject styles inline -->
<svelte:options css="injected" />
```

Context : createContextbe

Creates a [get, set] pair of functions to share data between parent and child components without prop drilling. Parent sets the context, any child anywhere in the tree can get it. Preferred over setContext/getContext as it is type-safe and requires no keys.

----> src/routes/+page.svelte

```svelte
<script>
    // Import the setUserContext function
    import { setUserContext } from '$lib/context/index.svelte.js';
    import Child from '$lib/components/Child.svelte'; // Import the Child component
    setUserContext({ name: 'world' }); // Set the user context
</script>

<Child /> <!-- Render the Child component -->
```

File path : src/lib/components/Child.svelte

```svelte
<script>
    import { getUserContext } from '$lib/context/index.svelte.js'; // Import the user context
    const user = getUserContext(); // Get the user context
</script>

<h1>hello {user.name}</h1> <!-- Render the user name -->
```

File Path src/lib/context/index.svelte.js

```js
import { createContext } from 'svelte'; // import the `createContext` function

export const [getUserContext, setUserContext] = createContext(); // Create a context
```

Context : With state

Global $state works fine for sharing state across components, but in SvelteKit SSR, it can leak data between users. Context is SSR-safe — each request gets its own data. Use $state for client-only, context for SSR.
Store $state in context so multiple child components share the same reactive state.

----> src/routes/+page.svelte

```svelte
<script>
    import { setCounter } from '$lib/context/index.svelte'; // Import the setCounter
    import Child from '$lib/components/Child.svelte';       // Import the Child component
    let coutner = $state({ count: 0 }); // Initialize count state
    setCounter(coutner); // Set the counter
</script>

<!-- Increment & reset count state -->
<button onclick={() => (coutner.count += 1)}>Increment</button>
<button onclick={() => (coutner.count = 0)}>Reset</button>

<Child /> <Child /> <Child />
```

File path : src/lib/components/Child.svelte

```svelte
<script>
    import { getCounter } from '$lib/context/index.svelte'; // Import the getCounter
    const counter = getCounter(); // Get the counter
</script>

<p>{counter.count}</p> <!-- Render the count -->
```

File Path src/lib/context/index.svelte.js

```js
import { createContext } from 'svelte'; // import createContext

export const [getCounter, setCounter] = createContext(); // Create the counter context
```

About : Lifecycle hooks

In Svelte 5, the component lifecycle consists of only two parts: Its creation and its destruction. Everything in-between — when certain state is updated — is not related to the component as a whole; only the parts that need to react to the state change are notified. This is because under the hood the smallest unit of change is actually not a component, it's the (render) effects that the component sets up upon component initialization. Consequently, there's no such thing as a "before update"/"after update" hook.

Lifecycle hooks : onMount

Does not run on SSR. If a function is returned, it Runes on unmount.

-----> src/routes/+page.svelte

```
<script>
    import { onMount } from 'svelte';

    // run component on mount
    onMount(() => {
        console.log('component mounted');

      // option cleanup on unmount
        return () => {
            console.log('component unmounted');
        };
    });
</script>
```

Lifecycle hooks : onDestroy

Schedules a callback to run immediately before the component is unmounted.

Out of onMount, beforeUpdate, afterUpdate and onDestroy, this is the only one that Runes inside a server-side component.

-----> src/routes/+page.svelte

```
<script>
    import { onDestroy } from 'svelte'; // import onDestroy

    // onDestroy is called when the component is being destroyed
    onDestroy(() => {
        console.log('the component is being destroyed');
    });
</script>

<!-- Render the component -->
<h1>Hello Svelte learner</h1>
```

Lifecycle hooks : tick

While there's no "after update" hook, you can use tick to ensure that the UI is updated before continuing. tick returns a promise that resolves once any pending state changes have been applied, or in the next microtask if there are none, wait for DOM update

-----> src/routes/+page.svelte

```
<script>
    import { tick } from 'svelte'; // Import the tick function

    // Use the tick function run after the DOM is updated
    $effect(() => {
        tick().then(() => {
            console.log('DOM updated');
        });
    });
</script>
```

$state : Only for reactive variables

For large objects that are only reassigned, use $state.raw — better performance.

----> src/routes/+page.svelte

```
// ✅ normal reactive state
let count = $state(0);

// ✅ large object — only reassigned, use $state.raw
let apiResponse = $state.raw({});
apiResponse = await fetch('/api/data').then(r => r.json());
```

$derived : Compute from state

Always use $derived not $effect for computed values.

----> src/routes/+page.svelte

```
// ✅ do this
let square = $derived(num * num);

// ❌ don't do this
let square;
$effect(() => { square = num * num; });
```

$props : Treat as changeable

Always use $derived for values that depend on props.

----> src/routes/+page.svelte

```
let { type } = $props();

// ✅ do this
let color = $derived(type === 'danger' ? 'red' : 'green');

// ❌ don't do this — won't update if type changes
let color = type === 'danger' ? 'red' : 'green';
```

Events : Use directly on elements

----> src/routes/+page.svelte

```
<!-- ✅ do this -->
<button onclick={() => {}}>click me</button>
<svelte:window onkeydown={handler} />
<svelte:document onvisibilitychange={handler} />

<!-- ❌ don't do this -->
<button on:click={() => {}}>click me</button>
```

Each blocks : Always use keys

----> src/routes/+page.svelte

```
<!-- ✅ keyed — better performance -->
{#each items as item (item.id)}
  <p>{item.name}</p>
{/each}

<!-- ❌ no key — avoid -->
{#each items as item}
  <p>{item.name}</p>
{/each}
```

CSS : Style child components

-----> src/routes/+page.svelte

```
<!-- ✅ CSS custom properties — preferred -->
<Child --color="red" />

<!-- Child.svelte -->
<h1>Hello</h1>
<style>
  h1 { color: var(--color); }
</style>

<!-- ✅ :global — only when child is from a library -->
<div>
  <Child />
</div>
<style>
  div :global { h1 { color: red; } }
</style>
```

Context : Use createContext

-----> src/routes/+page.svelte

```
// ✅ do this
import { createContext } from 'svelte';
export const [getUser, setUser] = createContext();

// ❌ don't do this
import { setContext, getContext } from 'svelte';
setContext('user', user);
```

Avoid : Legacy

-----> src/routes/+page.svelte

```
<!-- ❌ Legacy → ✅ Modern -->

<!-- state -->
let count = 0           →   let count = $state(0)

<!-- derived -->
$: square = num * num   →   let square = $derived(num * num)

<!-- props -->
export let name         →   let { name } = $props()

<!-- events -->
on:click={handler}      →   onclick={handler}

<!-- slots -->
<slot />                →   {#snippet children()}{/snippet}
                            {@render children()}

<!-- actions -->
use:action              →   {@attach action}

<!-- class directive -->
class:active={bool}     →   class={{ active: bool }}
```